

Type Safety via Hoare Logic with Separation Types

WENHUA LI DARIUS FOO QUANG-TRUNG TA WEI-NGAN CHIN

Type safety has traditionally been ensured through crafted type systems operating under the motto “*well-typed programs cannot go wrong*”. However, modern demands push type systems beyond basic safety guarantees — toward greater memory safety (e.g., Rust), stronger data structure invariants (e.g., Haskell’s GADTs) and broader typability (e.g., MLStruct). In this paper, we present a foundational Hoare logic framework for type-safety verification that unifies three novel ingredients: (i) *separation* types, inspired by separation logic, to support flow-sensitive type mutation and must-aliasing; (ii) *type predicates* for expressing rich data structure invariants; and (iii) a disciplined distinction between Err (runtime errors) and Abrt (compile-time errors); enabling a refined motto — *well-typed programs must never abort*.

Our framework supports path-sensitive and flow-sensitive type specifications via case specifications and separation types, and handles a broad spectrum of scenarios from weaker type specifications that tolerate runtime errors to stronger ones that eliminate them entirely. We formalize our Hoare rules and prove soundness in Rocq, implement a prototype verifier called *TypeHL* in OCaml, and evaluate it on a suite of benchmark programs. Practicality is maintained by interpreting types as a Boolean algebra, reducing subtyping to emptiness checking.

ACM Reference Format:

Wenhua Li Darius Foo Quang-Trung Ta Wei-Ngan Chin . 2026. Type Safety via Hoare Logic with Separation Types. 1, 1 (March 2026), 32 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Type systems are being asked to do more than ever. Beyond the classic guarantee that well-typed programs do not go wrong, modern languages demand richer properties: memory safety through ownership and borrowing (Rust), stronger data structure invariants through generalized algebraic data types (GADTs in OCaml and Haskell), broader typability through semantic subtyping (MLsub [12], MLstruct [28]), and low-latency execution through modal resource management (OxCaml [25, 32]). Dynamically-typed languages face the same pressure from a different direction, as gradual typing retrofits (e.g. Hack [26] and Typed Racket [33]) attempt to recover static guarantees incrementally. Yet the foundational framework for reasoning about type safety has not kept pace: each new demand tends to be met by a bespoke type system extension, making it difficult to determine what type-safety guarantee each extension provides, whether two extensions are compatible, or how they can be combined into a single system.

In this paper, we argue that Floyd-Hoare logic provides exactly the right foundation for a unified treatment of type safety. Hoare logic is inherently modular: the type property of each function is captured by a pre/post specification that can be verified and reused independently. It is flow-sensitive by design, tracking how types evolve through a program rather than assigning a single static type to each variable. It is also expressive enough to accommodate both statically- and dynamically-typed programs within the same framework, by adjusting the strength of the specifications rather than changing the underlying logic. The key question is whether type signatures — the concise, familiar

Author’s address: Wenhua Li Darius Foo Quang-Trung Ta Wei-Ngan Chin.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

XXXX-XXXX/2026/3-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

form of type information — can be faithfully and conveniently represented as Hoare-style pre/post specifications. We show that they can, and that the resulting framework supports strictly more than standard type systems.

We now illustrate these points through a series of examples, building from simple type signatures to flow-sensitive specifications for heap-mutating programs. Consider four functions from an ML-like language: a (+) function to add two integers; a conversion function from Int to its Str counterpart; a polymorphic (!) operator to dereference a mutable Ref; and an update function to replace the stored value with a new value of type T.

```
(+)      : (Int, Int) → Int
str_of_int : Int → Str
(!)      : ∀T. Ref(T) → T
update   : ∀T. (Ref(T), T) → ()
```

While type signatures are concise, it is almost as easy to write their type specifications using pre/postconditions in Hoare logic. For the above examples, we can obtain the corresponding type specifications by suitably naming the inputs v^* and output r of each function using the precondition clause $\text{req}[v^*]$ and the postcondition clause $\text{ens}[r]$, respectively, as shown below.

```
(+)      : req[x, y] x: Int ∧ y: Int ens[r] r: Int
str_of_int : req[x] x: Int ens[r] r: Str
(!)      : ∀T. req[x] x: Ref(T) ens[r] r: T
update   : ∀T. req[x, u] x: Ref(T) ∧ u: T ens[r] r: ()
```

For the examples above, each type signature of the form $t_1 \rightarrow t_2$ can be directly translated into a type specification of the form $\text{req}[x] x:t_1 \text{ ens}[r] r:t_2$, where x and r are freshly named. So far, the gain over type signatures is mainly notational. The real advantage of Hoare logic emerges when type signatures fall short: they cannot track global variables, and they are flow-insensitive, assigning a single type to each variable regardless of how it evolves through the program. Both limitations matter for the following example, which uses a global variable g whose type changes from $g \mapsto \text{Ref}(\text{Int})$ before each `stateful_add` call to $g \mapsto \text{Ref}(\text{Str})$ after it — a pattern that is common in dynamically-typed programs.

```
stateful_add : Int → Str
stateful_add y = update(g, str_of_int(y+!g)); !g
```

To capture this kind of flow-sensitive type change, a pure type $g:\text{Ref}(\text{Any})$ is too weak — it loses track of the precise type stored at g after each update. We instead introduce the separation type $g \mapsto \text{Ref}(t_1)$, which carries full ownership of the heap location at g and can be updated to a different type $g \mapsto \text{Ref}(t_2)$ as the program evolves. Coupled with pre/post type specification, we can now provide more precise and comprehensive type specifications for both `stateful_add` and `update`, as shown below.

```
stateful_add : req[y] y: Int ∧ g ↦ Ref(Int)
               ens[r] r: Str ∧ g ↦ Ref(Str)
update : ∀T. case[x, u] { x: Ref(T) ∧ u: T ens[r] r: () ;
                       x ↦ Ref(_) ∧ u: Any ens[r] x ↦ Ref(u) ∧ r: () }
```

For the update method, we use a case specification to capture both the flow-insensitive pure-type scenario $x:\text{Ref}(T)$, and the flow-sensitive separation-type scenario $x \mapsto \text{Ref}(_)$. In this paper, we develop a Hoare logic framework for type safety in which *separation* types (with must-aliasing) provide flow-sensitivity, while flow-insensitive *pure* types (with arbitrary aliasing) are used whenever type mutation is not required.

Our main contributions are:

- We propose a Floyd-Hoare logic approach to type safety verification, and introduce separation types to support must-aliasing and flow-sensitive type mutation. Used alongside flow-insensitive pure types, these two ingredients provide a unified framework with increased coverage.
- Our approach supports path-sensitivity via case specifications, and strengthened preconditions via type predicates, subsuming GADTs.
- We distinguish recoverable runtime errors (Err), from compile-time errors (Abt). The latter never occurs for well-typed programs.
- We design a small core language, define its operational semantics, and construct a logic of separation and pure types. We formalize our Hoare rules in Rocq and prove soundness.
- We implement a prototype our verifier called *TypeHL* in OCaml and evaluate it on a suite of programs.

Paper structure. The rest of our paper is organized as follows. Sec. 2 covers the novel aspects of our type logic via examples and can be read independently as an overview. Sec. 3 presents a core language and the type logic that supports expressive type specifications for both statically-typed and dynamically-typed languages. Sec. 4 presents forward-style Hoare rules for type safety, building on Sec. 3. Sec. 5 defines the semantics of our type logic and formalises the Hoare rules in Rocq, and establishes soundness. Sec. 6 describes the *TypeHL* prototype and its evaluation. Sec. 7 shows how our type predicate framework subsumes both GADT and Liquid Types. Sec. 8 discusses related work. The appendices cover additional features and extensions.

2 IMPROVEMENTS TO TYPE-SAFETY VIA HOARE LOGIC

The following four sections introduce for novel ingredients of our proposal. Case specifications (Sec. 2.1) give path-sensitivity. Separation types (Sec. 2.2) provide the expressive power needed for flow-sensitive heap reasoning. The Err/Abt error hierarchy (Sec. 2.3) makes error coverage explicit, verifiable and customisable to the precision required by the caller. Type predicates (Sec. 2.4) capture data structure invariants. Although independent contributions, all four compose coherently, as we summarise at the end of Sec. 2.

2.1 Semantic Subtyping, Path-Sensitivity and Case Specifications

Semantic-based subtyping interprets each type t as a set of values, and defines subtyping $t_1 <: t_2$ via set containment $\text{Set}(t_1) \subseteq \text{Set}(t_2)$. This allows types to be composed freely using a Boolean algebra:

$$t := t \vee t \mid t \wedge t \mid \neg t \mid \dots$$

Consider the ML-style function below, which ML's type system rejects because unification fails when it tries to combine the Int output of one branch with the Str output of the other:

```
(* A function returning Int or Str *)
inc_if_int(x) = match x of { Int -> x+1;
                          _   -> "Not supported!" }
```

Semantic subtyping resolves this: one valid formulation assigns the output type as $\text{Int} \vee \text{Str}$ yielding $\text{inc_if_int} : \text{Any} \rightarrow \text{Int} \vee \text{Str}$. The most precise set-theoretic type is an intersection of arrow types, $(\text{Int} \rightarrow \text{Int}) \wedge (\neg \text{Int} \rightarrow \text{Str})$ [6], which our case specification below directly mirrors.

```
(* Type signature of inc_if_int *)
inc_if_int : Any → Int ∨ Str
```

Our preferred formulation is a case specification:

```
(* Our proposed type specification using case-specs *)
inc_if_int : case[x] { x:Int   ens[r] r:Int;
                      x:¬Int  ens[r] r:Str }
```

Case specifications differ from intersection types in one crucial respect: disjointness is a verified obligation, not a programmer convention. The type checker rejects any case specification whose guards overlap, making the specification exhaustive and non-redundant by construction. A programmer writing the intersection type $(\text{Int} \rightarrow \text{Int}) \wedge (\text{Any} \rightarrow \text{Int} \vee \text{Str})$ receives no such check — the overlapping cases pass silently. This enforcement is what makes the otherwise clause and the `Abt` discipline of Sec. 2.3 well-defined.

2.2 Flow-Sensitivity and Separation Types

Many languages support pass-by-value-result parameters (e.g. `inout` parameters in Swift) that allow variable arguments to be mutated by a method call. Dynamically-typed languages go further, allowing the type of a variable to change arbitrarily, as follows:

```
inc_transform (inout x) = x := str_of_int(x+1)
```

One might suggest $\text{Int} \vee \text{Str} \rightarrow ()$ as the type signature of `inc_transform`, but this is still not type-safe: $\text{Int} \vee \text{Str}$ is weaker than the `Int` precondition required by the argument of `(+)` method. Standard type systems, including Swift's, are flow-insensitive: any type assigned to `x` must be simultaneously valid as the argument to `(+)` (requiring `Int`) and as the final stored value after `str_of_int` (requiring `Str`). The join $\text{Int} \vee \text{Str}$ satisfies neither constraint precisely. Flow-sensitivity resolves this by tracking the type of `x` at each program point: $x:\text{Int}$ in the pre-state (entering the body), and $x':\text{Str}$ in the post-state (after the update), so every sub-expression sees the correct type.

```
inc_transform : req[x] x:Int ens[r] x':Str ∧ r:()
```

where x' in the postcondition denotes the mutated type of parameter `x`.

Tracking type mutation for heap memory locations is considerably harder, because we must account for must-aliases among heap locations. Linear (uniqueness) types guarantee that each heap location has a single reference and no aliases, which rules out aliasing but also rules out programs where aliasing is intentional. We instead introduce a **separation type**, written $x \mapsto T$, inspired by separation logic, which carries full ownership of the heap location at address `x` and supports must-aliasing explicitly. A must-alias is a statically known alias: `y` is a must-alias of `x` if `y=x` is guaranteed at that program point. Must-aliasing under a separation type is the key to supporting strong updates — changing the stored type of a heap location — since every alias is known and will consistently observe the new type. Pure types, by contrast, permit arbitrary aliasing (any number of unknown references), making strong updates unsound.

Why must-aliasing matters: a preview. Before giving the formal specifications, we highlight the key feature that distinguishes separation types from uniqueness types. Consider the swap function:

```
swap(x, y) = let v1=!x in let v2=!y in
             update(x, v2); update(y, v1)
```

There are (at least) three meaningful scenarios for swap: (i) flow-insensitive access where types are not mutated; (ii) two distinct heap locations whose contents are exchanged; and (iii) `x` and `y` are aliases of the same heap location, in which case swap is a no-op. A uniqueness type system cannot handle case (iii) at all, since uniqueness requires exactly one reference per location. Separation types accommodate must-aliases explicitly via the singleton type $y:\{x\}$ (meaning `y=x`), making them strictly more expressive. We return to this example below after introducing the formal separation type specifications.

Formal specifications. Using separation types, we can provide stronger, flow-sensitive specifications for the core heap operations. We also introduce spatial conjunction $*$ to capture pairs of disjoint heap locations:

Here Any_P denotes the universe of pure types — types that allow arbitrary aliasing, as opposed to separation types which carry exclusive ownership. Any_P is given a precise definition in Sec. 5.1.

We use $x:t$ in two ways. For a value variable x , it asserts x has type t . For a type variable T , the bounded quantifier $\forall T:R$ means T ranges over subtypes of R ; subtyping is otherwise written $T <: R$. Thus $x:\text{Ref}(T)$ is a pure type assertion on a value, while $x \mapsto \text{Ref}(y)$ asserts heap ownership where $y : \text{Any}$ (a type or a singleton $\{v\}$).

```

mkRef: req[x] x:Any ens[r] r $\mapsto$ Ref(x)
(!):  $\forall T:\text{Any}_P, q:\text{Any}. \text{case}[m] \{ m:\text{Ref}(T) \text{ ens}[r] r:T; \\ m \mapsto \text{Ref}(q) \text{ ens}[r] m \mapsto \text{Ref}(q) \wedge r:q \}$ 
update:  $\forall T:\text{Any}_P. \text{case}[m, v] \{ m:\text{Ref}(T) \wedge v:T \text{ ens}[r] r:(); \\ m \mapsto \text{Ref}(\_) \wedge v:\text{Any} \text{ ens}[r] m \mapsto \text{Ref}(v) \wedge r:() \}$ 

```

The precondition of `mkRef` is maximally weak (Any covers both pure and separation types), while its postcondition is maximally strong. The case specifications of `update` and `(!)` cover both the flow-insensitive pure-type scenario and the flow-sensitive separation-type scenario.

The two-case structure of `update` directly reflects the pure vs. separation type distinction and illustrates the flexibility of our framework. In Case 1, the caller holds a pure type $m:\text{Ref}(T)$: only a type-consistent write is permitted (Definition 2.1), and the postcondition returns $()$ without changing the type of m . In Case 2, the caller holds full ownership $m \mapsto \text{Ref}(_)$: a type mutation is permitted (Definition 2.2), and the postcondition reflects the flow-sensitive update, yielding $m \mapsto \text{Ref}(v)$. A single method specification thus serves both regimes, with the case guard determining which applies at each call site.

DEFINITION 2.1 (TYPE-CONSISTENT MUTATION FOR PURE TYPES). *A write $\text{update}(x, v)$ to a location $x:\text{Ref}(T)$ in pre-state Δ is type-consistent if $v:T$. Type mutation — changing the stored type from T to $T_2 \neq T$ — is disallowed under pure types: because arbitrary aliasing means other references to x would observe an inconsistent type, only type-consistent writes are permitted.*

DEFINITION 2.2 (TYPE MUTATION FOR SEPARATION TYPES). *Type mutation at a location $x \mapsto \text{Ref}(T)$ is the replacement of the stored type T with a distinct type T_2 , effected by a write $\text{update}(x, v)$ where $v:T_2$. The postcondition reflects the updated type: $x \mapsto \text{Ref}(T_2)$. Type mutation is safe precisely because full ownership of x ensures there are no pure-heap aliases to x — every alias is a known must-alias that observes the updated type consistently.*

Irreversible weakening. A key design choice is that separation types can be irreversibly weakened to pure types, but not vice versa:

$$x \mapsto T \wedge T <: A \wedge A <: \text{Any}_P \Rightarrow_{\text{weaken}} x:A$$

Once ownership is surrendered via weakening, the right to type-mutate is permanently lost. Concretely: $r \mapsto \text{Ref}(T)$ (from `mkRef`) can be weakened to $r : \text{Ref}(T)$ when type mutation is no longer required; the reverse is disallowed since other aliases may already exist. Separation types are designed to manage ownership and permission to access disjoint regions of memory. In particular, separation types support must-aliasing, while pure types allow arbitrary aliasing. We therefore require each separation type $x \mapsto T$ to be mutually disjoint from the pure type $x:T$, as captured by the following axiom:

$$x \mapsto T \wedge x:T \Leftrightarrow \text{false}$$

REMARK 2.1 (TWO HEAP PARTITIONS). *The separation heap gives exclusive ownership $x \mapsto T$: full rights to read, mutate the type, and track must-aliases via singleton types $\{z\}$. The pure heap gives shared access $x:T$: no ownership, type-consistent writes only. The axiom $x \mapsto T \wedge x:T \Leftrightarrow \text{false}$ enforces disjointness. The weakening lemma $x \mapsto T \Rightarrow x:A$ is the only crossing point – irrevocable.*

Must-aliasing: the swap example. We can now give the case specification for swap:

```
swap :  $\forall A : \text{Any}_P, a, b : \text{Any}. \text{ case } [x, y] \{$ 
   $x : \text{Ref}(A) \wedge y : \text{Ref}(A) \text{ ens}[r] \ r : () ;$ 
   $x \mapsto \text{Ref}(a) * y \mapsto \text{Ref}(b) \text{ ens}[r] \ x \mapsto \text{Ref}(b) * y \mapsto \text{Ref}(a) \wedge r : () ;$ 
   $x \mapsto \text{Ref}(a) \wedge y : \{x\} \text{ ens}[r] \ x \mapsto \text{Ref}(a) \wedge r : () \}$ 
```

The first case handles flow-insensitive pure-type access. The second case handles two distinct locations (denoting disjoint heap ownership) whose values are swapped. The third case uses the singleton type $y:x$ to express that y must-alias x (i.e. $y=x$); swapping is then a no-op and the separation type is preserved. This third case is simply inexpressible with uniqueness types, confirming that separation types are strictly more expressive (see Appendix D.1).

REMARK 2.2 (WELL-FORMEDNESS BY CONSTRUCTION). *A separation type $x \mapsto T$ may contain pure components; the converse does not hold. The weakening lemma $x \mapsto T \wedge T <: A \wedge A <: \text{Any}_P \Rightarrow x : A$ is the only separation-to-pure crossing rule; both T and the target A must be pure subtypes of Any_P . No converse rule exists.*

REMARK 2.3 (SINGLETON TYPES AND ALIASING). *A singleton type $\{z\}$ denotes the one-element set $\{z\}$ — the most precise type for a value, naming its exact witness. The aliasing consequences follow the precision of the assertion: (i) $\{z\} <: \text{Any}_S$: full separation ownership, no unknown aliases, type mutation sound; (ii) $x:\{z\}$ with $\{z\} <: \text{Any}_P$: witness known but pure heap, unknown aliases may mutate independently, type mutation unsound; (iii) $x:A$ with $A <: \text{Any}_P$: no witness named, arbitrary unknown aliases, type mutation unsound. Type mutation is sound only in case (i); in cases (ii) and (iii) the location resides in the pure heap where aliased access is permitted, making type mutation unsound.*

REMARK 2.4 (MIXED SCENARIOS VIA WEAKENING). *The spec omits mixed cases where one parameter is a pure type and the other a separation type, specifically: $x:\text{Ref}(A) \wedge y \mapsto \text{Ref}(b)$ and $x \mapsto \text{Ref}(a) \wedge y:\text{Ref}(B)$. A caller willing to surrender ownership can apply the weakening lemma to reduce to Case 1; otherwise the implicit otherwise clause applies, yielding Abt .*

2.3 A Disciplined Error Hierarchy

We begin with a concrete example that motivates our error hierarchy before presenting it in full. Consider the head function on lists:

```
head :  $\forall T. \text{List}(T) \rightarrow T$ 
```

Two distinct kinds of failure can arise. If head is called with the wrong type of argument — say an Int — the type checker can detect this statically and flag an Abt error — a distinguished value representing compile-time type-errors that, according to Definition 5.3, must never appear in the postcondition of a well-typed program. The program is not well-typed and must not be allowed to run.

If head is called with a Nil value (the correct type, but an empty list), a runtime error occurs: the program has gone wrong in a way that was not caught at compile time, and the result is best represented by an Err value that propagates through subsequent computations.

Traditional type systems (e.g. OCaml, Scala) conflate runtime errors with the bottom type $\perp$, treating them as outside the type system entirely. This conflation loses precision: $\perp$ denotes non-termination or unreachability — a function with postcondition $r:\perp$ is one that never returns. A

runtime error, by contrast, is a propagatable value that flows into subsequent computations. By internalising runtime errors as the first-class value `Err` in our type lattice, our framework can track whether and where errors arise and propagate — something $\perp$ -based treatments cannot express. We instead distinguish `Err` and $\perp$ explicitly.

Three type specifications for `head`. Our framework allows the programmer to choose the level of error specification appropriate for their context:

```

301 head :  $\forall T. \text{ req}[x] \ x:\text{List}(T) \ \text{ ens}[r] \ r:\text{TVerErr}$ 
302 head :  $\forall T. \text{ case}[x] \ \{ \ x:\text{Cons}(T, \_) \ \text{ ens}[r] \ r:T \ ;$ 
303            $\quad \quad \quad x:\text{Nil} \ \text{ ens}[r] \ r:\text{Err} \ \}$ 
304 head :  $\forall T. \text{ req}[x] \ x:\text{Cons}(T, \_) \ \text{ ens}[r] \ r:T$ 

```

The first two specifications treat `head(Nil)` as a tracked runtime error; the third eliminates it as a compile-time error. The second and third are not comparable in terms of strength: they make different trade-offs between permissiveness and static guarantees. Together, the three specifications illustrate the range of expressiveness available in our framework — a range that eludes most traditional type systems without dependent types or type qualifiers.

The error hierarchy. We design three categories of error types:

- `Err`: a runtime error value that may be passed as an argument to method calls. Subtypes include `unDef` (for uninitialised values) and `null` (for null pointer dereferences). These are tolerated in programs and do not trigger compile-time errors until they are actually used.
- `Exc`: an exception, further split into `ChExc` (checked; must be tracked explicitly by our type system) and `UnExc` (unchecked; treated as a subtype of valid types and may be omitted from specifications). Exception handling is discussed in Appendix C.
- `Abt` (and $\top$): a compile-time type error. Our type-safety verification framework must prove `Abt` and $\top$ unreachable for any well-typed program.

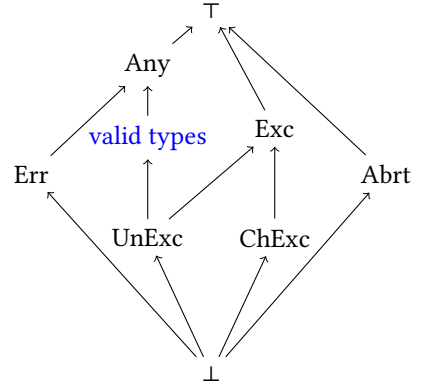


Fig. 1. Lattice of Types

Consider map:

```

329 map(f, xs) = match xs of { Nil -> Nil;
330                        Cons(y, ys) -> Cons(f(y), map(f, ys)) }

```

Its type specification with case specs is:

```

333 map:  $\forall A, B. \text{ case}[f, xs] \ \{ \ f:\text{Any} \wedge xs:\text{Nil} \ \text{ ens}[r] \ r:\text{Nil};$ 
334        $\quad \quad \quad f:A \rightarrow B \wedge xs:\text{Cons}(A, \text{List}(A)) \ \text{ ens}[r] \ r:\text{Cons}(B, \text{List}(B));$ 
335        $\quad \quad \quad (\_) \ \text{ ens}[r] \ r:\top \}$ 

```

The first two cases capture valid type specifications; the otherwise clause $(_)$ triggers a compile-time error if the inputs `f` and `xs` do not match either case. Explicitly specifying $\top$ or `Abt` flags compile-time type errors. The disjointness of case guards (Sec. 2.1) is essential here: an input satisfying multiple guards would induce an intersection type, and without disjointness the verifier would need to perform a proof search to determine which case applies. Enforced disjointness eliminates this need for proof search.

REMARK 2.5 (THE $\perp$ TYPE AND NON-TERMINATION). *Following standard convention, $\perp$ denotes the empty set of values and is a subtype of every type ($\forall t. \perp <: t$). As a postcondition, $r:\perp$ asserts that no value is ever returned – the computation diverges. As a precondition, $x:\perp$ asserts that the input can never exist – an unreachable branch. Crucially, in our framework $\perp$ is reserved for these two standard roles. Runtime errors are not conflated with $\perp$: they are internalised as the first-class value `Err`, enabling precise error tracking that $\perp$ -based treatments cannot provide. For example: $(\text{req}[x] \ x : \text{Any} \ \text{ens}[r] \ r:\perp)$ is the correct specification of a diverging function – since loop never returns, no value r is ever produced, and $\perp$ accurately reflects this.*

REMARK 2.6 (UnExc AND $\perp$). *UnExc should not be conflated with $\perp$: UnExc is inhabited (it classifies raised exceptions), whereas $\perp$ has no inhabitants and denotes non-termination.*

2.4 Pure and Separation Type Predicates

Modern functional languages, such as ML and Haskell, support algebraic data types (ADTs). Four simple examples are shown below.

```
data Color    = Red   | Black
data Nat      = Zero  | Succ(Nat)
data List(T) = Nil    | Cons(T, List(T))
data Tree(T) = Leaf   | Node(color, T, Tree(T), Tree(T))
```

We propose using type predicates to model algebraic data types. Like types, type predicates can be recursive, but are strictly more expressive. Each ADT can be encoded as a pure type predicate, as shown below.

```
pred x:List(T) = x:Nil  ∨ ∃ r. x:Cons(T,r) ∧ r>List(T)
pred n:Nat     = n:Zero ∨ ∃ r. n:Succ(r) ∧ r:Nat
pred x:Color   = x:Red  ∨ x:Black
pred x:Tree(T) = x:Leaf ∨ ∃ c,lt,rt. x:Node(c,T,lt,rt)
                    ∧ c:color ∧ lt:Tree(T) ∧ rt:Tree(T)
```

Type predicates are interpreted inductively as least fixed points – their recursive unfolding is assumed to terminate (well-foundedness), a condition stated precisely in Remark 3.1 at the end of this section. (We assume here a strict language. If a lazy language is used, we will need to make use of an interpretation via greatest fixed points.)

To capture data structures with stronger invariant properties, Haskell and OCaml also support a restricted form of dependent types known as Generalised Algebraic Data Types (GADTs). Here, their type parameters may be separately instantiated, depending on the data constructors used – as illustrated by the red/black balanced tree below.

```
data RBTREE :: Color → Nat → * → * where
  Leaf      :: RBTREE Black Zero a
  RedNode   :: RBTREE Black h a → a
              → RBTREE Black h a → RBTREE Red h a
  BlackNode :: RBTREE cl h a → a
              → RBTREE cr h a → RBTREE Black (Succ h) a
```

Rather than adopting GADT data types, we propose that our framework use type predicates for capturing data structures with strong invariant properties. Two examples are shown below.

```
pred x:List(T,s) = x:Nil ∧ s:Zero
                  ∨ ∃ m. x:Cons(T,r) ∧ r>List(T,m) ∧ s:Succ(m)
pred x:RBTREE(T,c,h) = x:Leaf ∧ c:Black ∧ h:Zero
```



```

393       $\vee \exists h_1, lt, rt. \ x: \text{Node}(c, T, lt, rt) \wedge lt: \text{RBTREE}(T, \_, h_1)$ 
394       $\wedge rt: \text{RBTREE}(T, \_, h_1) \wedge c: \text{Black} \wedge h: \text{Succ}(h_1)$ 
395       $\vee \exists r, lt, rt. \ x: \text{Node}(c, T, lt, rt) \wedge lt: \text{RBTREE}(T, \text{Black}, h)$ 
396       $\wedge rt: \text{RBTREE}(T, \text{Black}, h) \wedge c: \text{Red}$ 

```

The predicate $x:\text{List}(T, s)$ uses an extra parameter s to track the length of the list. The predicate $x:\text{RBTREE}(T, c, h)$ uses c to denote the colour of the red/black tree and h to capture its black-height, and enforces that the black-height is always balanced. These richer predicates are related to their base-type counterparts by the following lemmas.

```

402      lemma  $\forall x, T. (\exists n. \ x: \text{List}(T, n)) \iff x: \text{List}(T)$ 
403      lemma  $\forall x, T, c, h. \ x: \text{RBTREE}(T, c, h) \implies x: \text{Tree}(T)$ 

```

Stable pure and separation types. With length-indexed predicates in hand, we now define stability for pure types (Definition 3.1) and separation types (Definition 3.2). Stability of type predicates themselves is deferred to Sec. 3.2 (Definitions 3.1-3.2), where the necessary formal vocabulary is introduced.

DEFINITION 2.3 (STABLE PURE TYPE). *A pure type t is stable under heap mutation if it cannot be invalidated by any type-consistent mutation (Definition 2.1). Formally, for any heaps h, h' related by a type-consistent mutation with respect to the current pre-state Δ , $s, h \models x : t$ implies $s, h' \models x : t$. Singleton types $y:x$ (asserting $y=x$) [8, 28] are trivially stable: they live in the pure constraint π and express only an equality between program or logical variables (including existentially quantified $\exists v^*$ in Δ) — a fact that is independent of the heap and therefore unaffected by any heap mutation.*

DEFINITION 2.4 (STABLE SEPARATION TYPE). *A separation type $x \mapsto T$ is stable if the content type T is a stable pure type. Full ownership of x is guaranteed by the separation type discipline itself, so the non-trivial requirement is on the content type T : for example, $x \mapsto \text{Ref}(\text{Int} \vee \text{Str})$ is stable because $\text{Int} \vee \text{Str}$ is stable, whereas $x \mapsto \text{Ref}(T_2)$ may not be stable if T_2 can be invalidated by aliased writes to its fields. Unlike a stable pure type, a stable separation type additionally permits type mutation of its content via an explicit update, since full ownership ensures no pure-heap alias to x exists and every must-alias observes the updated type consistently.*

It is also possible to support richer data structures via separation type predicates which may capture heap nodes exclusively owned by the predicates, such as $x \mapsto \text{MList}(T)$:

```

428      data MList(T) = Nil | Cons(T, Ref(MList(T)))
429      pred  $x \mapsto \text{MList}(T) = x: \text{Nil} \vee \exists r, r_1. \ x \mapsto \text{Cons}(T, r) * r \mapsto \text{Ref}(r_1) * r_1 \mapsto \text{MList}(T)$ 

```

Such mutable lists can be used to construct fully owned mutable structures, including circular and doubly-linked lists — a domain well covered by two decades of research in separation logic. We propose to make these features available within the type system to support greater memory safety.

Precise definitions of stable pure and separation type predicates, in terms of the formal type logic, are deferred to Definitions 3.1 and 3.2 in Sec. 3.2, once the required vocabulary — immutable constructors, heap assertions, ownership — has been formally established.

The well-foundedness assumption underlying all type predicate definitions is stated formally as Remark 3.1 in Sec. 3.2, once the formal vocabulary is in place.

A more detailed comparison of type predicates with GADTs and liquid types is in Chapter 7.

2.5 A Combined Example

We close with a single example that exercises all four ingredients simultaneously, previewing how they compose before the formal rules. Consider `rm_head`, which dereferences a heap-allocated list reference, returns the head element if the list is non-empty, and returns `Err` otherwise:

```
rm_head(m) = let xs = !m in
  match xs of { Nil -> err ;
               Cons(y,_) -> update(m, tail(xs)); y }
```

We use the length-indexed predicate $\text{List}(T, s)$ — where s is a Nat tracking the length — to give a precise three-case specification. $s:\text{Zero}$ means the list is empty; $s:\text{Succ}(s1)$ means it has at least one element.

```
rm_head :  $\forall T, l, s, s1.$ 
  case[m] { (* Case 1: pure-type access, possible runtime Err *)
    m:Ref(l) ens[r] r:TVErr ;
    (* Case 2: sep type, empty list . rm_head returns Err *)
    m $\mapsto$ Ref(l) $\wedge$ !List(T, s) $\wedge$ s:Zero ens[r] m $\mapsto$ Ref(l)  $\wedge$  r:Err;
    (* Case 3: sep type, non-empty list . flow-sensitive mutation *)
    m $\mapsto$ Ref(l) $\wedge$ !List(T, s1) $\wedge$ s:Succ(s1) ens[r]  $\exists l1. m \mapsto \text{Ref}(l1) \wedge l1:\text{List}(T, s1) \wedge r:T }$ 
```

This specification uses all four ingredients:

- Case specifications (Sec. 2.1) — three pairwise-disjoint guards covering pure-type access, the empty-list separation case, and the non-empty-list separation case.
- Separation types (Sec. 2.2) — Cases 2 and 3 use $m \mapsto \text{Ref}(l)$ to track ownership of the heap location. In Case 3 the postcondition reflects the flow-sensitive update: m now points to the tail $l1$ of the original list.
- Error typing (Sec. 2.3) — Case 1 tolerates a runtime `Err` explicitly via $r : T \vee \text{Err}$; Case 2 returns `Err` precisely when the list is known to be empty, rather than conflating it with $\perp$ or an unchecked exception.
- Type predicates (Sec. 2.4) — $\text{List}(T, s)$ is a length-indexed recursive predicate. The Nat index s distinguishes `Zero` (empty) from `Succ(s1)` (non-empty), giving the verifier the information it needs to prove Cases 2 and 3 disjoint and to track the length decrease in the postcondition of Case 3.

Stability of type predicates. This example also illustrates precisely where pure-type stability holds and where it breaks down.

$l:\text{List}(T, s)$ is stable under mutation: `List` is built from immutable `Cons` constructors, so no mutation can alter the chain of nodes or their count. The length index s is fixed at construction time, and any alias that reads the list observes the same s .

By contrast, $m:\text{Ref}(\text{List}(T, s))$ is not stable: the `Ref` cell can be updated by any alias to point to a different list with a different length s' , invalidating the assertion on s . This is precisely why Case 1 uses the weaker $m:\text{Ref}(\text{List}(T))$ without a length index — the length cannot be stably tracked through a pure `Ref` type in the presence of aliased mutation.

Cases 2 and 3 resolve this by taking full ownership via $m \mapsto \text{Ref}(l) \wedge l:\text{List}(T, s)$: since there are no pure-heap aliases to m , `Ref(l)` is fully owned, allowing a precise update in the postcondition of Case 3. The verifier *TypeHL* checks this specification compositionally: the three cases are verified independently under their respective preconditions, and the separation type in Cases 2 and 3 is consumed and re-emitted by the `Call` rule (Sec. 4); ownership is handled by spec composition, ensuring no ownership is lost across the update call.

3 CORE LANGUAGE AND A LOGIC FOR TYPES

This section has two parts. The first (Sec. 3.1) presents the syntax of the core language: a small strict higher-order functional language with immutable constructors and a single mutable heap type `Ref`. The second (Sec. 3.2) defines the type logic — the grammar of states, heap assertions, pure constraints, and type specifications — that Sec. 4 uses to state and verify type-safety properties. The notation `req/ens` used informally throughout Sec. 2 is given its formal definition here.

3.1 Core Language

We consider a strict higher-order functional language with immutable data constructors with the exception of the `Ref` type. The key design choices are as follows. To support both static and dynamic typing, we organise data constructors and primitive types into a subtype lattice with `Any` as the supertype of all valid types, covering both user-defined and primitive types. Each `let` binding introduces an immutable variable. Functions and data constructors are always fully applied. Function definitions and lambda abstractions are always given explicit type specifications. We also provide an alternative style for function specifications, $\lambda x^* r. \Phi[r]$, so that `case` and `ens` constructs do not need to explicitly track the parameters/result. For simplicity, we omit `inout` parameters and exceptions.

The syntax is summarised in Figure 2. We briefly explain each syntactic category. A program (`Prog`) is a sequence of definitions. Each definition (`Defn`) binds a function name to a body expression together with a type specification $\lambda x^* r. \Phi$. The full expression language (`Full`) includes the standard constructs — values, `let`-bindings, constructor applications, function calls, type casts (`t`), `e`, and `match` — and is the language the programmer writes. The core expression language (`Core`) is the normalised form used for Hoare-style reasoning: constructor arguments and function arguments are always simple variables, and the `match` scrutinee is always a variable. Base types (`Base`) include singleton types, primitive types, constructor types, predicate types, and the special types `Err`, `Abrt`, `Any`, $\perp$, and $\top$. Types (`Type`) extend base types with separation types ($\mapsto$ base), function types, and Boolean combinations $\wedge$, $\vee$, $\neg$. Patterns (`Pattern`) mirror types and are used in `match`; note that $\top$ and $\perp$ are excluded from patterns since the former trivially holds and testing for the latter is equivalent to the halting problem.

(Prog)	$prog ::= def_1 ; \dots ; def_n$
(Defn)	$def ::= f\ x_1 \dots x_n = e\ \text{where}\ f :: \lambda x_1 \dots x_n r. \Phi[r]$
(Full)	$e ::= val \mid \text{let } x:t=e_1 \text{ in } e_2 \mid C(e_1 \dots e_n) \mid (f\ e_1 \dots e_n) \mid (t)\ e$ $\mid \text{match } e \{p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n\}$
(Value)	$val ::= x \mid c \mid \text{err} \mid \text{abrt} \mid \lambda x^*. e :: \lambda x^* r. \Phi[r]$
(Core)	$e ::= val \mid \text{let } x=e_1 \text{ in } e_2 \mid C(x_1 \dots x_n) \mid (f\ x_1 \dots x_n) \mid (t)\ x$ $\mid \text{match } x \{p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n\}$
(Base)	$base ::= x \mid \{x\} \mid \{c\} \mid \text{Bool} \mid \text{Int} \mid \text{Str} \mid \text{Ref}(\dots) \mid \dots \mid \text{Err} \mid \text{Abrt} \mid \text{Any} \mid \perp \mid \top$ $\mid C(t_1 \dots t_n) \mid \text{pred}(t_1 \dots t_n)$
(Type)	$t ::= base \mid \mapsto base \mid t_1 \rightarrow t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t$
(Pattern)	$p ::= c \mid \text{Bool} \mid \text{Int} \mid \text{Str} \mid \dots \mid \text{Err} \mid \text{Abrt} \mid \text{Any} \mid C(x_1 \dots x_n) \mid \text{pred}(_)$ $\mid _ \rightarrow _ \mid p_1 \wedge p_2 \mid p_1 \vee p_2 \mid \neg p \mid _$ $c \in \mathbb{B} \cup \mathbb{Z} \cup \dots \quad x, f, r \in \text{vars}$

Fig. 2. Syntax of Full and Core Language with Types

The `Full` $\Leftrightarrow$ `Core` preprocessing works by inserting eager casts to `Any` at every constructor argument, function argument, and `match` scrutinee. `Any` is the supertype of all normal runtime

types but does not include *Abt* or *ChExc*. The casts are the propagation points: if any sub-expression produces *Abt* or *ChExc*, the cast short-circuits and propagates the exceptional value outward, so the core *let* construct never needs to handle these cases directly. The preprocessing rules are:

$$\begin{aligned}
 C(e_1, \dots, e_n) &\Leftrightarrow \text{let } \{x_i:\text{Any}=e_i\}_{i=1}^n \text{ in } C(x_1, \dots, x_n) \\
 f\ e_1 \dots e_n &\Leftrightarrow \text{let } \{x_i:\text{Any}=e_i\}_{i=1}^n \text{ in } f\ x_1 \dots x_n \\
 \text{match } e \{ \dots \} &\Leftrightarrow \text{let } x:\text{Any}=e \text{ in match } x \{ \dots \} \\
 \text{let } x:t=e_1 \text{ in } e_2 &\Leftrightarrow \text{let } x = (t)\ e_1 \text{ in match } x \{ \text{Abt} \rightarrow x \mid _ \rightarrow e_2 \} \\
 (t)\ e &\Leftrightarrow \text{let } x=e \text{ in } (\text{let } x_1=(t)\ x \text{ in } x_1)
 \end{aligned}$$

Pattern overlap and desugaring. For ease of programming, we allow the patterns in each match construct to overlap, relying on top-down processing to give an unambiguous semantics. Before Hoare-style reasoning begins, overlapping patterns are desugared into disjoint form. For example, $\text{match } x \{ p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2 \mid p_3 \rightarrow e_3 \}$ becomes $\text{match } x \{ p_1 \rightarrow e_1 \mid p_2 \wedge \neg(p_1) \rightarrow e_2 \mid p_3 \wedge \neg(p_1 \vee p_2) \rightarrow e_3 \}$. The Hoare rules can then deal with complex patterns formed from $\wedge$, $\vee$, and $\neg$. To support efficient runtime execution, complex type patterns are further translated to simpler patterns using the rules below:

$$\begin{aligned}
 \text{match } x \{ \dots \mid p_1 \vee p_2 \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p_1 \rightarrow e \mid p_2 \rightarrow e \mid \dots \} \\
 \text{match } x \{ \dots \mid p_1 \wedge p_2 \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p_1 \rightarrow \text{match } x \{ p_2 \rightarrow e \mid \dots \} \mid \dots \} \\
 \text{match } x \{ \dots \mid \neg p \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p \rightarrow \text{match } x \{ \dots \} \mid _ \rightarrow e \}
 \end{aligned}$$

3.2 Type Logic and Specifications

We now define the type logic that underpins the Hoare rules of Sec. 4. The central object is a type state Δ , which pairs a heap assertion σ capturing ownership of heap locations with a pure constraint π capturing type and equality facts about variables. The grammar is given in Figure 3.

$$\begin{aligned}
 (\text{State}) \quad \Delta &::= \bigvee_{i=1}^n \exists v^* \cdot \sigma_i \wedge \pi_i \\
 (\text{Heap}) \quad \sigma &::= \text{emp} \mid x \mapsto t \mid \sigma_1 * \sigma_2 \\
 (\text{Pure}) \quad \pi &::= x:t \mid x_1=x_2 \mid \pi_1 \wedge \pi_2 \mid \pi_1 \vee \pi_2 \mid \neg \pi \mid \exists v^* \cdot \pi \\
 (\text{Spec}) \quad \Phi &::= \text{ens}[r] \Delta \mid \text{case } \{ \Delta_1 \Phi_1; \dots; \Delta_n \Phi_n \} \mid \forall x^* \cdot \Phi
 \end{aligned}$$

Fig. 3. Type Logic Specification

Each syntactic category plays a distinct role. A state Δ is a disjunction of existentially quantified heap-pure pairs; disjunction arises naturally from path-sensitive case analysis. A heap assertion σ is either empty (*emp*), a separation type $x \mapsto t$ asserting full ownership of the location at x with content type t , or a separating conjunction $\sigma_1 * \sigma_2$ asserting disjoint ownership. A pure constraint π records type memberships $x:t$, equalities, and Boolean combinations thereof; it does not assert ownership. Note that we support $\vee$, $\wedge$ and $\neg$ at both the type language and the assertion language. We support union types $T_1 \vee T_2$ in both the type language and the assertion language, with the pure-type equivalence $r:T_1 \vee T_2 \Leftrightarrow r:T_1 \vee r:T_2$ holding as a derived rule. A specification Δ is either a simple postcondition $\text{ens}[r] \Delta$, a case specification with pairwise-disjoint guards ($\forall i \neq j. \Delta_i \wedge \Delta_j = \text{false}$), or a universally quantified specification. The shorthand $\text{req } \Delta_1 \text{ ens}[r] \Delta_2$ abbreviates $\text{case } \{ \Delta_1 \Rightarrow \text{ens}[r] \Delta_2 \}$, formalising the notation used throughout Sec. 2.

For completeness, every case specification is implicitly extended with an otherwise clause ($_ \Rightarrow \text{ens}[r] \text{ r:T}$) to flag any input not covered by the stated cases as a compile-time *Abt* error. This is consistent with the disjointness discipline of Sec. 2.1: because the stated guards are pairwise disjoint, the otherwise clause is well-defined and covers precisely the remaining inputs.

The vocabulary established by Figure 3 — in particular, immutable constructors (values built by the Base grammar that do not involve $\mapsto$), pure sub-components (occurrences of π in a state), and ownership (heap assertions σ) — allows us to give the formal definitions of stable type predicates that were deferred from Sec. 2.4.

DEFINITION 3.1 (STABLE PURE TYPE PREDICATE). *A pure type predicate $\text{pred } x : P(v^*)$ is stable if (i) its unfolding uses only pure types and immutable constructors (Base terms not involving $\mapsto$), and (ii) every $\text{Ref}(T')$ in the unfolding has T' a stable pure type. Stability of recursive occurrences follows from these two conditions by structural induction on the unfolding.*

A further refinement is needed for separation type predicates, since they may contain pure sub-components that remain exposed to aliased mutation even though the separation part itself is protected by heap assertion σ . These pure sub-components must therefore themselves be stable.

DEFINITION 3.2 (STABLE SEPARATION TYPE PREDICATE). *A separation type predicate $\text{pred } x \mapsto P(v^*)$ is well-formed and stable if: (i) full ownership of x is maintained — guaranteed automatically by the heap assertion $x \mapsto$ in Figure 3; and (ii) every pure sub-component in the unfolding of P is stable: bare pure sub-components $y:T$ must be stable pure types, and pure type predicate sub-components $y:Q(v^*)$ must be stable pure type predicates in the sense of Definition 3.1. Unlike pure predicates, a stable separation predicate additionally permits type mutation — the predicate may be updated to $x \mapsto P'(v^*)$ via an explicit update, tracked in the postcondition — provided the updated predicate also satisfies conditions (i) and (ii).*

Definitions 3.1 and 3.2 require well-founded predicate unfoldings, as does Definition 2.3 for pure types appearing inside them. We make this assumption explicit.

REMARK 3.1 (WELL-FOUNDEDNESS AND CO-INDUCTION). *As is standard in Hoare logic for strict (eager) languages, all type predicates — both pure and separation — are assumed to be **well-founded**: their recursive unfolding always terminates. For lazy languages, this assumption must be relaxed via co-inductive reasoning. Extending our framework to lazy languages via co-inductive type predicates is a direction for future work.*

The $\vdash \wedge$ operator, used in the Hoare rules of Figure 4, is a special conjunction that performs case analysis on both separation and pure types. Intuitively, $(\Delta \vdash \wedge \pi)$ refines the state Δ by the pure constraint π , yielding $\Delta \wedge \pi$ when no heap ownership is involved. When a separation type σ is present, $\vdash \wedge$ additionally performs bi-abductive heap entailment to extract the residual frame Δ_s : $\Delta \vdash \wedge \sigma \wedge \pi \Leftrightarrow \Delta_s \wedge \pi \wedge \pi_0$ where $\Delta \wedge \pi_0 \vdash \sigma \Rightarrow \sigma_s$. This allows the Frame rule of Sec. 4.1 to thread ownership through function calls without losing heap context.

4 HOARE LOGIC RULES FOR TYPE-SAFETY

This section presents the forward Hoare rules that constitute the core of our type-safety verification framework. Sec. 4.1 introduces the two structural rules (Conseq and Frame) and explains how a type specification is composed with a program state via the $(:)$ operator. Sec. 4.2 presents the rules for each expression form. Sec. 4.3 gives a worked example showing how the rules interact.

4.1 Structural Rules and Specification Composition

We present a forward-style Hoare rule of the form $\{ \Delta_{\text{pre}} \} e \{ \text{ens}[r] \Delta_{\text{post}} \}$: given an input type state Δ_{pre} , the rule computes the strongest post-state Δ_{post} resulting from evaluating e , binding the result and its type to r . Two structural rules apply to every expression form. Conseq allows the pre-state to be weakened and the post-state to be strengthened via entailment ($\vdash$ denotes intuitionistic implication). Frame allows a disjoint heap context Δ to be threaded through any

expression unchanged, provided the expression does not touch the locations asserted in Δ — this is the separation logic Frame rule lifted to our type logic.

$$\frac{\Delta_1 \vdash \Delta_3 \{ \Delta_3 \} e \{ \text{ens}[r] \Delta_4 \} \Delta_4 \vdash \Delta_2}{\{ \Delta_1 \} e \{ \text{ens}[r] \Delta_2 \}} \text{Conseq}$$

$$\frac{\{ \Delta_1 \} e \{ \text{ens}[r] \Delta_2 \}}{\{ \Delta_1 * \Delta \} e \{ \text{ens}[r] \Delta_2 * \Delta \}} \text{Frame}$$

Specification composition. The Hoare rules for function calls rely on a type specification being provided for each method, using $f::\lambda x^* r. \Phi$. The specification must be in case-spec form and is merged with the pre-state Δ via the operator $\Delta; \Phi$, defined by the two reduction rules below. Intuitively, $\Delta; \Phi$ applies the specification Φ in the context of the current state Δ : for each case guard Δ_1 , the operator checks how much of Δ satisfies Δ_1 (via $\vdash \wedge$), and the remainder becomes the post-state for that branch. Branches whose guard is inconsistent with Δ produce an Abrt outcome.

$$\begin{aligned} \Delta; \text{case } \{ \Delta_1 \Phi_1; \dots; \Delta_n \Phi_n \} &\Rightarrow ((\Delta \vdash \wedge \Delta_1); \Phi_1) \vee \dots \vee ((\Delta \vdash \wedge \Delta_n); \Phi_n) \\ \Delta; \text{ens}[r] \Delta_2 &\Rightarrow \Delta * \Delta_2 \end{aligned}$$

We introduce a special conjunction $\vdash \wedge$ that performs case analysis on both separation and pure types. With pure types this operator is equivalent to standard conjunction: $(\Delta \vdash \wedge \pi) \Leftrightarrow (\Delta \wedge \pi)$. In the presence of a separation type σ , $\vdash \wedge$ performs bi-abductive heap entailment to obtain a residual frame Δ_s :

$$\frac{\Delta \wedge \pi_0 \vdash \sigma \rightsquigarrow \Delta_s}{\Delta \vdash \wedge \sigma \wedge \pi \Leftrightarrow \Delta_s \wedge \pi \wedge \pi_0}$$

Example (specification composition). Consider a call $(f \ x)$ where f has specification $\Phi \equiv \text{case}[x] \{ x : \text{Intens}[r] \ r : \text{Int}; x : \neg \text{Intens}[r] \ r : \text{Abrt} \}$. If the pre-state is $\Delta \equiv x : \text{Int}$, then $\Delta; \Phi$ yields $x : \text{Int} \wedge r : \text{Int}$ (the first branch matches, the second is vacuous). If instead $\Delta \equiv x : \text{Any}$, then $\Delta; \Phi$ yields $(x : \text{Int} \wedge r : \text{Int}) \vee (x : (\text{Any} \wedge \neg \text{Int}) \wedge r : \text{Abrt})$, signalling a possible Abrt from inputs not known to be Int.

4.2 Rules for Expression Forms

The remaining rules in Figure 4 handle each expression form in the core language. We describe each rule in turn.

Rule commentary. Var records that the result r equals the variable x via the singleton type $r : x$. Constr introduces fresh type variables $T_1..T_n$ for the field types and produces a separation type $r \mapsto C(T_1..T_n)$, giving the caller full ownership of the freshly allocated node. Val handles literal constants, err, and abrt, each assigned their canonical type via $\text{ty}(v)$. Call applies specification composition $(\Delta; \Phi)$ as described in Sec. 4.1. Cast produces two branches: one where the cast succeeds ($x:t$, result is x itself) and one where it fails ($\neg(x:t)$, result is Abrt); the preprocessing of Sec. 3.1 ensures Abrt propagates outward. Let threads the post-state of e_1 (with r_1 substituted by x) into e_2 . Match distributes the pre-state across branches by conjoining each pattern guard, collecting post-states in disjunctive form. Lambda uses the auxiliary checking judgement $\{ \Delta \} e \{ \downarrow \Phi \}$ (Figure 5) to verify that the body e satisfies the supplied specification Φ , then records the closure's specification in the post-state.

While we support higher-order functions in our Hoare rules, we currently restrict function type parameters to flow-insensitive pure types, since we rely on a two-stage pre/post specification without higher-order constraints. This allows such functions to be treated as pure functions

$$\begin{array}{c}
\frac{\text{fresh } r}{\{\Delta\} x \{ \text{ens}[r] \Delta \wedge r : x \}} \text{Var} \quad \frac{\text{fresh } r, T_1..T_n \quad \Delta_1 \equiv (\Delta \wedge \bigwedge_{i=1}^n x_i : T_i)}{\{\Delta\} C(x_1..x_n) \{ \text{ens}[r] \Delta_1 \wedge r \mapsto C(T_1..T_n) \}} \text{Constr} \\
\\
\frac{\text{fresh } r \quad v ::= c \mid \text{err} \mid \text{abrt}}{\{\Delta\} v \{ \text{ens}[r] \Delta \wedge r : \text{ty}(v) \}} \text{Val} \quad \frac{\text{fresh } r \quad \Delta \vdash f :: \lambda x^* r. \Phi}{\{\Delta\} f(x^*) \{ \text{ens}[r] \Delta; \Phi \}} \text{Call} \\
\\
\frac{\text{fresh } r \quad \Delta_1 \equiv (\Delta \wedge (x:t) \wedge r:x) \vee (\Delta \wedge \neg(x:t) \wedge r:\text{Abt})}{\{\Delta\} (t) x \{ \text{ens}[r] \Delta_1 \}} \text{Cast} \\
\\
\frac{\{\Delta\} e_1 \{ \text{ens}[r_1] \Delta_1 \} \quad \{ [r_1 := x] \Delta_1 \} e_2 \{ \text{ens}[r] \Delta_2 \}}{\{\Delta\} \text{let } x = e_1 \text{ in } e_2 \{ \text{ens}[r] \Delta_2 \}} \text{Let} \\
\\
\frac{\{\Delta \wedge (x:p_1 \vee x \mapsto p_1)\} e_1 \{ \text{ens}[r] \Delta_1 \} \cdots \{\Delta \wedge (x:p_n \vee x \mapsto p_n)\} e_n \{ \text{ens}[r] \Delta_n \}}{\{\Delta\} \text{match } x \{ p_1 \rightarrow e_1 \mid \cdots \} \{ \text{ens}[r] \bigvee_{i=1}^n \Delta_i \}} \text{Match} \\
\\
\frac{\text{fresh } f \quad v^* = \text{vars}(\Delta) - \text{vars}(e) \cup \{x^*\} \quad \{\exists v^*. \text{Pure}(\Delta)\} e \{ \downarrow \Phi \}}{\{\Delta\} \lambda x^*. e :: \lambda x^* r. \Phi \{ \text{ens}[f] \Delta \wedge f :: \lambda x^* r. \Phi \}} \text{Lambda}
\end{array}$$

Fig. 4. Forward Hoare Rules on Expressions for Type Safety

without any type mutation. Simple parameter variables are earmarked to allow singleton types (see Appendix A.1), which reduces the use of quantified type variables.

4.3 Worked Example

To illustrate how the rules interact, we trace the derivation for the `inc_transform` function from Sec. 2.2: `inc_transform(inout x) = x := str_of_int(x + 1)`, whose specification is $\text{req}[x] \ x : \text{Int} \ \text{ens}[r] \ x' : \text{Str} \wedge r : ()$. Starting from pre-state $\Delta \equiv x : \text{Int}$, the `Let` rule sequences the sub-expressions; each intermediate `Call` rule applies the specification of `(+)` (resp. `str_of_int`, `update`) via $(\Delta; \Phi)$, threading ownership through. The final post-state $x' : \text{Str} \wedge r : ()$ matches the declared postcondition, so the specification is verified. The separation-type case (where $x \mapsto \text{Ref}(T)$ is held) follows the same derivation pattern but additionally uses the `Frame` rule to thread the ownership $x \mapsto \text{Ref}(T)$ through sub-expressions that do not touch x .

$$\begin{array}{c}
\frac{\{\Delta * \Delta_1\} e \{ \downarrow \Phi_1 \} \cdots \{\Delta * \Delta_n\} e \{ \downarrow \Phi_n \}}{\{\Delta\} e \{ \downarrow \text{case } \{ \Delta_1 \Phi_1; \cdots; \Delta_n \Phi_n \} \}} \text{Spec-Case} \\
\\
\frac{\{\Delta\} e \{ \text{ens}[r] \Delta_1 \} \quad \Delta_1 \vdash \Delta_2}{\{\Delta\} e \{ \downarrow \text{ens}[r] \Delta_2 \}} \text{Spec-Ens}
\end{array}$$

Fig. 5. Checking Hoare Rules for Type Specification

The checking judgement $\{\Delta\} e \{ \downarrow \Phi \}$ is used when processing lambda abstractions (`Lambda` rule): the specification Φ is supplied as an input, and the rule verifies that the body e satisfies it. `Spec – Case` dispatches on each case guard, prepending the guard Δ_i to the pre-state and checking the body against the corresponding Φ_i . `Spec – Ens` checks that the forward post-state Δ_1 entails the declared postcondition Δ_2 .

5 SOUNDNESS OF OUR TYPE SPECIFICATION FRAMEWORK

This section presents the soundness theorem for the Hoare rules of Sec. 4, establishes the connection between Hoare triples and standard function types, and summarises the Rocq formalisation. Sec. 5.1 defines the semantics of the type logic. Sec. 5.2 states and proves the main soundness theorem. Sec. 5.3 summarises the Rocq development and case studies.

5.1 Semantics of the Type Logic

We have formalized our Hoare logic with pure and separation types to ensure type safety in Rocq and proved its soundness [1].

The semantics is given by a logical relation (Figure 6). Rather than using explicit stores and name manipulation, we represent variables using Rocq variables directly. In keeping with semantic subtyping, types are shallowly embedded as predicates of type $\text{val} \rightarrow \text{Prop}$, so proving a type assertion $v:t$ amounts to proving the proposition $t(v)$ by semantic reasoning. This shallow embedding means logical connectives for building types are lifted directly from their propositional counterparts, and subtyping reduces to implication: $t_1 <: t_2 \equiv \forall v.v : t_1 \Rightarrow v : t_2$. The encoding supports dependent singleton types (Appendix A.1) and (co)inductive types via Rocq's own (co)inductive types. Step-indexing is not required because the encoding is stateless and involves no non-(co)inductive types.

$s, h \models x:c$	<i>iff</i> $s(x)=c$	$s, h \models x_1:x_2$	<i>iff</i> $s(x_1)=s(x_2)$
$s, h \models x_1=x_2$	<i>iff</i> $\exists l.s(x_1)=s(x_2)=l$		
$s, h \models \Delta_1 \vee \Delta_2$	<i>iff</i> $s, h \models \Delta_1$ <i>or</i> $s, h \models \Delta_2$		
$s, h \models \sigma \wedge \pi$	<i>iff</i> $\exists h_1, h_2. h = h_1 \text{oh}_2 \wedge s, h_1 \models \sigma \wedge s, h_2 \models \pi$		
$s, h \models \sigma_1 * \sigma_2$	<i>iff</i> $\exists h_1, h_2. h = h_1 \text{oh}_2 \wedge s, h_1 \models \sigma_1 \wedge s, h_2 \models \sigma_2$		
$s, h \models \pi_1 \vee \pi_2$	<i>iff</i> $s, h \models \pi_1$ <i>or</i> $s, h \models \pi_2$		
$s, h \models \pi_1 \wedge \pi_2$	<i>iff</i> $s, h \models \pi_1$ <i>and</i> $s, h \models \pi_2$		
$s, h \models \neg \pi$	<i>iff</i> <i>not</i> $(s, h \models \pi)$		
$s, h \models \Delta_1 * \Delta_2$	<i>iff</i> $s, h \models (\sigma_1 * \sigma_2) \wedge (\pi_1 \wedge \pi_2)$ <i>where</i> $\Delta_i = \sigma_i \wedge \pi_i$		
$s, h \models x:t$	<i>iff</i> $\exists l.s(x)=l$ <i>and</i> $s, h \models l:t$		
$s, h \models l:\text{prim}$	<i>iff</i> $l \in \text{prim}$ <i>where</i> $\text{prim} = \text{Int} \mid \text{Str} \mid \text{Err} \mid \dots$		
$s, h \models l \rightarrow C(t_1, \dots, t_n)$	<i>iff</i> $h = \{l \rightarrow C(l_1, \dots, l_n)\}$ <i>and</i> $\bigwedge_{i=1}^n s, h \models l_i:t_i$		
$s, h \models l:C(t_1, \dots, t_n)$	<i>iff</i> $h(l) = C(l_1, \dots, l_n)$ <i>and</i> $\bigwedge_{i=1}^n s, h \models l_i:t_i$		
$s, h \models l:\text{pred}(t^*)$	<i>iff</i> $l:\text{pred}(t^*) = \Delta$ <i>and</i> $s, h \models \Delta$		
$s, h \models l:\neg t$	<i>iff</i> <i>not</i> $(s, h \models l:t)$		
$s, h \models l:t_1 \rightarrow t_2$	<i>iff</i> $\forall v.(s, h \models v:t_1 \rightarrow s, h \models l(v):t_2)$		

Fig. 6. Semantics of Type Logic Formulae

Several clauses deserve comment. The clause for $\sigma \wedge \pi$ splits the heap into two disjoint parts h_1 and h_2 : h_1 satisfies the heap assertion σ (ownership) and h_2 satisfies the pure constraint π (no ownership). The clause for $l \rightarrow C(\dots)$ requires h to consist of exactly the single cell l , reflecting full ownership; by contrast, $l:C(\dots)$ only requires l to point to a C -node somewhere in h , permitting aliasing. This is the semantic counterpart of the syntactic distinction between separation types and pure types established in Sec. 2.2. The clause for pred unfolds the predicate definition, connecting type predicates (Sec. 2.4) to their set-theoretic meaning.

5.2 Soundness Theorem

The connection between Hoare triples and standard function types is established by defining a modified function arrow $\rightarrow'$ in terms of Hoare triples and showing it coincides with the standard arrow:

$$\{ \Delta_1 \} e \{ \text{ens}[r] \Delta_2 \} \quad \text{iff} \quad (\Delta_1 \implies \forall r, e \rightsquigarrow r \implies r \models \Delta_2) \quad (1)$$

$$f:t_1 \rightarrow' t_2 \quad \text{iff} \quad \forall v, \{ v:t_1 \} f(v) \{ \text{ens}[r] r:t_2 \} \quad (2)$$

Defining a modified function arrow $\rightarrow'$ in terms of a standard Hoare triple (which is also defined extensionally, using a big-step relation), we can see that it is equivalent (a lifting of $\iff$) to the regular function arrow from Fig. 6.

LEMMA 5.1 (AGREEMENT BETWEEN FUNCTION ARROWS). $t_1 \rightarrow t_2 \equiv t_1 \rightarrow' t_2$.

This lemma shows that the behaviours of well-typed programs are exactly those of programs specified by Hoare triples: the two notions of function type are definitionally equivalent. It is proved by unfolding both definitions and using the big-step operational semantics.

The soundness of the Hoare rules is stated as follows.

THEOREM 5.2 (SOUNDNESS). *Given $\{ \Delta_1 \} e \{ \text{ens}[r] \Delta_2 \}$, $\forall s, h, s', h', v$ if $s, h \models \Delta_1$ holds and $s, h, e \rightsquigarrow s', h', v$, then $s' + [(r, v)], h' \models \Delta_2$.*

PROOF. The proof proceeds by induction on the derivation of the Hoare triple, with a compatibility lemma for each rule in Figure 4. The use of big-step semantics in Theorem 5.2 is deliberate: big-step semantics guarantees that if an expression terminates it reaches its postcondition, and non-terminating expressions satisfy the postcondition vacuously. This means a single theorem simultaneously captures both type preservation (the type of the result matches the postcondition) and progress (no well-typed, terminating expression gets stuck). Traditional type systems typically require two separate theorems — one for preservation and one for progress — using small-step semantics.

□

Definition 5.3 (Well-typed programs never abort). e is well-typed under satisfiable Δ_1 if and only if $\{ \Delta_1 \} e \{ \text{ens}[r] \Delta_2 \}$ and Δ_2 does not contain Abt.

This definition formalises the refined motto of Sec. 2.3: well-typed programs must never abort. Note that Err (runtime errors) may appear in Δ_2 — they are tolerated. Only Abt is forbidden, since Abt represents a compile-time type error that the verification framework must rule out entirely.

5.3 Rocq Formalization and Case Studies

The formalisation is available together with the paper. It contains 66 lemmas over 1,153 lines of Rocq and builds on the Separation Logic Foundations library[9]. Since [9] supports only flow-insensitive pure types, we extended its library to include our flow-sensitive separation types. The only axiom used pervasively is the law of excluded middle, required in proofs concerning negated types.

Table 1 summarises the case studies. Each row lists: LoC (lines of definition and proof, as a proxy for difficulty), and which features the example exercises — recursion (Rec), higher-order functions (HO), subtyping (Sub), and singleton types (Sing).

The case studies are intentionally small — their purpose is to validate the proof rules and demonstrate feature coverage, not to stress-test scalability. Larger-scale evaluation is left to future work. The tail and apply examples are notable: tail exercises subtyping without recursion, while apply exercises higher-order functions and singleton types simultaneously, confirming that the two features compose correctly in the Rocq development.

Case study	Lemmas	LoC	Rec	HO	Sub	Sing
id (Appendix A.1)	2	29	✗	✗	✗	✓
tail (Section 1)	5	110	✗	✗	✓	✗
apply (Section 1)	3	113	✗	✓	✗	✓
length (Appendix A.2)	1	49	✓	✗	✗	✗

Table 1. Case studies

6 IMPLEMENTATION AND EVALUATION.

This section describes the TypeHL prototype. Sec. 6.1 presents the architecture. Sec. 6.2 reports the evaluation.

6.1 Implementation

Architecture. *TypeHL*¹ is implemented in OCaml and operates as a *lightweight* forward compositional analyser: it requires no SMT solver, no abstract interpretation framework, and no heavyweight constraint solving — only a specification map, a forward analyser, and a simple entailment checker. It consists of three main components working in sequence. First, a *specification map* T records the type specification $\lambda x^*. r. \Phi$ for each function defined in the program; specifications are populated as functions are processed in definition order, so that recursive calls are handled by a fixed-point iteration on T . Second, a *forward analyser* $\text{post}(\Delta_0, T, e)$ takes a pre-state Δ_0 and the current map T , and computes the strongest post-state Δ_1 for a method body e by recursively applying the proof rules of Fig. 4. Third, an *entailment checker* decides whether the computed post-state Δ_1 entails the declared postcondition Δ_q , i.e. whether $\Delta_1 \models \Delta_q * \text{emp}$; if so, the specification is marked as verified in T and reused for subsequent callers.

Connection to the Hoare rules. *TypeHL* implements the checking judgement $\{\Delta\} e \{\downarrow \Phi\}$ of Fig. 5, which drives the top-level verification loop. The rules of Fig. 5 decompose into two groups based on how faithfully they are implemented.

The following rules are implemented *directly*: VAR (singleton type $r : x$); VAL (canonical type $\text{ty}(v)$); CAST (two-branch split on $x : t$ and $\neg(x : t)$); LET (sequential composition by substitution); MATCH (per-branch guard conjunction); CALL (specification composition $\Delta ; \Phi$); CONSEQ (entailment check via the lightweight checker); and FRAME. The CONSTR rule is also direct: it introduces fresh type variables $T_1, \dots, T_n$ for field types and produces a separation type $r \mapsto C(T_1, \dots, T_n)$, giving the caller full ownership of the freshly allocated node.

The LAMBDA rule requires a heuristic: verifying the body e against the supplied specification Φ requires instantiating the universally quantified variables in Φ . *TypeHL* currently handles this by requiring all type variables to be explicitly provided in the annotation $\lambda x^*. r. \Phi$; no type variable inference is performed. This is a deliberate simplification — it keeps the proof search algorithm decidable and predictable — but it means *TypeHL* is not a type *inference* engine: it is a type *verifier* that checks user-supplied specifications against implementations.

Entailment checker. By design, *TypeHL*'s entailment checker is intentionally *lightweight*: it performs no SMT solving and invokes no external oracle. Concretely, it proceeds in two steps.

First, *heap matching*: it checks that every separation-type assertion $x \mapsto T$ appearing in the declared postcondition Δ_q is matched by a corresponding assertion in the computed post-state Δ_1 , consuming ownership tokens pairwise and verifying that the residual frame is **emp**.

¹*TypeHL* is publicly available at (hidden in support of anonymity)

Second, *subtype checking*: it verifies that every pure type assertion $x : t$ in Δ_q is entailed by Δ_1 , reducing subtyping to Boolean algebra checking.

The checker has one known limitation: it does not currently support arithmetic index constraints, such as those arising from the length-indexed predicate $List(T, n)$ when n ranges over integers. Discharging such constraints in general would require an SMT backend (e.g. via the theory of linear integer arithmetic, LIA). We identify adding an SMT backend as a direction for future work. For now, we rely on a set of pre-defined type predicates to solve simple constraints.

If the entailment check succeeds, the specification is marked as verified in T and its entry in the specification map is reused by all subsequent callers.

6.2 Evaluation

We evaluate *TypeHL* on three dimensions: (i) a baseline benchmark suite comparing *TypeHL* against standard type systems, in particular OCaml; (ii) a feature comparison against four systems; (iii) standard libraries and mutable data structure demonstrating practical coverage.

6.2.1 Baseline Benchmarks. Table 2 compares *TypeHL* against the OCaml type system on a suite of benchmark programs covering the four ingredients of Sec. 2: case specifications, separation types, error typing, and type predicates. Each row records: *LoC* (lines of code), *LoS* (lines of specification – one pre/post pair per line), *T* (verification time in seconds), and *TP* for the OCaml: ✓ means OCaml assigns an equivalent annotation; × means the program is not typeable in OCaml; – means OCaml can type the program but with strictly less precision than *TypeHL*. We discuss each group in turn.

Benchmark	<i>TypeHL</i>			OCaml	Benchmark	<i>TypeHL</i>			OCaml
	LoC	LoS	<i>T</i>			LoC	LoS	<i>T</i>	
basic	30	13	< 0.01	✓	map	2	3	< 0.01	✓
stateful_funs	4	3	< 0.01	×	tail	2	2	< 0.01	–
refs_pure	6	4	< 0.01	✓	length	3	2	< 0.01	–
refs_sep	6	5	< 0.01	×	length_generic	4	3	< 0.01	×
swap_pure	4	1	< 0.01	✓	shape_funs	6	2	< 0.01	×
swap_sep	5	3	< 0.01	×	singleton_type	6	3	< 0.01	–
RBTree_insert	12	8	< 0.01	–	RBTree_member	7	5	< 0.01	–

Table 2. A Comparison with OCaml type system.

Benchmarks where OCaml agrees (✓): These benchmarks use only flow-insensitive pure types without type mutation. OCaml assigns an equivalent annotation in each case, confirming that *TypeHL* correctly handles the baseline cases that standard type systems already cover. Their inclusion establishes that our framework is a conservative extension of standard typing, not a replacement.

Benchmarks where OCaml is less precise (–): OCaml assigns a valid but weaker type in each case. For *tail*, OCaml cannot express the path-sensitive distinction between Cons and Nil inputs; *TypeHL* uses a case specification to assign the precise output type *T* for the Cons case and *Err* for the Nil case. For *length*, OCaml returns Int for all inputs; *TypeHL* returns the singleton type 0 for the Nil case, which is strictly more informative. For *singleton_type*, *TypeHL* exploits singleton types to avoid a universally quantified type variable, giving a more precise specification than OCaml’s principal type. The *RBTree* examples concern GADT-style types. OCaml GADTs enforce the *RBTree* invariant by construction, but the invariant is implicit in the constructor structure – there is no way to state explicitly in a postcondition what colour or black-height the result of insert has. *TypeHL* instead expresses invariants as type predicates in pre/postconditions, making them inspectable, composable, and independent of the data definition.

Benchmarks where OCaml rejects (×): These benchmarks exercise features that are outside the scope of OCaml’s type system. *stateful_funs* uses a global variable whose type changes from `Ref(Int)` to `Ref(Str)` across a call — a flow-sensitive type mutation that OCaml’s flow-insensitive system cannot track. *refs_sep* and *swap_sep* require separation types and must-aliasing; in particular, the third case of *swap* — where both arguments alias the same heap location — is simply inexpressible in OCaml (see Sec. 2.2). *length_generic* and *shape_funs* use path-sensitive case specifications that dispatch on the runtime constructor of the argument (`Int`, `Str`, or `List`); OCaml’s unification-based inference rejects these because the branch output types cannot be unified into a single principal type.

6.2.2 Feature Comparison. Table 3 compares *TypeHL* against four systems, across five features. A ✓ denotes full support; × denotes no support; ~ denotes partial support (requires additional annotations or is restricted to specific program patterns).

TypeHL is the only system in the comparison with full support across all five features. OCaml supports none of them. Rust supports flow-sensitive types via its ownership and borrowing discipline, and partial separation type predicates via its borrow checker, but cannot express must-aliasing, path-sensitive case specifications, or the `Err/Abt` error hierarchy. Liquid Haskell supports partial path-sensitivity via refinement types, but has no notion of heap ownership, separation, or flow-sensitive type mutation.

Feature	TypeHL	OCaml	Rust	Liquid Haskell
Flow-sensitive types	✓	×	✓	×
Must-aliasing	✓	×	×	×
Path-sensitive specs	✓	×	×	~
Separation type predicates	✓	×	~	×
Err/Abt error hierarchy	✓	×	×	×

Table 3. Feature comparison. ✓ = full support; × = no support; ~ = partial support. *TypeHL* is the only system with full support across all features.

Function	LoC	LoS	T	Features exercised
List.append	4	2	<0.01	Recursion, type predicate <i>List(T)</i>
List.rev	5	2	<0.01	Recursion, accumulator pattern
List.map	2	3	<0.01	Higher-order, type predicate
List.filter	5	3	<0.01	Higher-order, path-sensitivity
List.fold_left	5	2	<0.01	Higher-order, recursion
List.length	4	2	<0.01	Length-indexed predicate <i>List(T, n)</i>
List.nth	5	3	<0.01	Err typing, case specification, type predicate
Option.map	3	3	<0.01	Path-sensitivity, Err typing
Option.bind	3	3	<0.01	Higher-order, path-sensitivity
Option.get	3	3	<0.01	Err typing, case specification
Queue.enqueue	5	4	<0.01	Separation types, must-aliasing, flow-sensitive mutation
Queue.dequeue	6	6	<0.01	Separation types, must-aliasing, Err typing

Table 4. Standard library. LoC = lines of code; LoS = lines of specification; T = verification time (seconds). All benchmarks verify in under 0.01 seconds.

6.2.3 Standard Library. To demonstrate coverage beyond toy programs, we reimplement a subset of the `List`, `Option` and mutable `Queue` modules in our core language and verify them with *TypeHL*. Table 4 summarises the results.

The standard library benchmarks collectively exercise all ingredients of Sec. 2 without arithmetic index constraints: recursion and accumulator patterns (append, rev), higher-order functions (map, filter, fold_left), path-sensitive error typing (nth), and singleton return types (length). Across all functions, *TypeHL* assigns strictly more precise types than typical type systems which collapse all branches into a single type. The two queue benchmarks demonstrate the three capabilities unique to *TypeHL*: separation type specifications tracking heap ownership via *, must-aliasing for the empty-queue via singleton type front : end (inexpressible in Rust or uniqueness types), and flow-sensitive postconditions reflecting type mutation at heap locations. All twelve benchmarks verify in under 0.01 seconds using only heap matching and subtype checking.

7 COMPARING WITH GADTS AND LIQUID TYPES

This section compares our type predicate framework with two closely related approaches: Generalised Algebraic Data Types (GADTs), which encode structural invariants into the type structure, and liquid types, which refine base types with logical predicates. We show that pure type predicates subsume both, and that separation type predicates go further still into heap ownership — a dimension neither GADTs nor liquid types can reach.

Pure type predicates are closely related to the recursive measures of Kawaguchi et al. [22], which encode structural invariants as terminating first-order functions over ADTs for use in liquid type refinements; separation type predicates are closely related to the user-defined inductive heap predicates of separation logic verifiers such as HIP [10]. Our contribution is to unify both within a single type logic, making them first-class types subject to the Boolean algebra of types, semantic subtyping, and a stability analysis that precisely distinguishes which predicates survive aliased mutation.

7.1 Comparison with GADTs

Type predicates in our framework strictly subsume GADTs in two important respects.

First, *type predicates can express relational index constraints that GADTs cannot*. GADT type parameters can only be equated to specific constructor-determined types; they cannot express *inequalities* or arithmetic relations between indices.

For example, a sorted list predicate with bounds:

```
pred x:SortList(T,lo,hi) = x:Nil ∨ ∃ r,m.
    x:Cons(v,r) ∧ v:T ∧ lo ≤ v ∧ v < hi ∧ r:SortList(T,v,hi)
```

requires the relational constraint $lo \leq v < hi$ between index values — something GADTs cannot express without full dependent types (e.g. Agda or Idris). Type predicates in our framework express this naturally as a logical formula.

Second, and more practically, type predicates do not require a **new algebraic data type for each new invariant**. With GADTs, every new invariant demands a completely new data type definition with new constructors encoding the invariant into the type structure. In our framework, the same underlying data representation — the same Cons and Nil constructors — can be given progressively richer type predicates without any change to the data definition itself:

```
pred x:List(T)           = ... (* plain list *)
pred x:List(T,n)         = ... (* length-indexed *)
pred x:SortList(T,lo,hi) = ... (* sorted, with bounds *)
```

All three predicates describe values built from the same constructors. This separation between *data representation* and *type invariant* means that existing code and data structures need not be refactored when a stronger invariant is required — only the specification changes. Separation type

predicates (e.g. $x \mapsto \text{List}(T)$) extend this further to heap ownership and spatial conjunction, which are entirely outside the scope of GADTs.

7.2 Comparison with Liquid Types

Pure type predicates are also closely related to liquid types (as in Liquid Haskell), which refine base types with logical predicates — for example, $\{v : \text{Int} \mid v > 0\}$ denotes positive integers. Like liquid types, our solution via type predicates also attaches logical invariants to values as formulas rather than encoding them into the type structure as GADTs do, and both allow the same underlying data representation to carry progressively stronger specifications without changing the data definition. There are, however, three important differences.

- Liquid types refine base types with first-order predicates over primitive values (integers, booleans); our pure type predicates are recursive, handling arbitrary inductively defined data structures such as lists, trees, and GADTs.
- Liquid types are restricted to decidable refinement logics (typically linear arithmetic) to keep SMT solving tractable; our predicates are not confined to a decidable fragment and can express arbitrary relational and structural invariants.
- Liquid types have no counterpart to separation type predicates: there is no notion of heap ownership or spatial conjunction in Liquid Haskell². Pure type predicates thus generalise liquid types from flat refinements over base values to recursive predicates over inductively defined structures, while separation type predicates extend this further to heap ownership with a mechanism that co-exists with pure types – dimension currently outside the scope of liquid types.

Though type predicates are not constrained to a decidable fragment, it is nevertheless quite easy to impose a set of restrictions that can guarantee decidability for type-checking. Appendix B.3 outlines restrictions that can be imposed on type predicates to support decidable type-checking.

8 RELATED WORKS

Semantic Subtyping. Semantic-based subtyping interprets each type as the set of values inhabiting it: $t_1 <: t_2$ holds when $\text{Set}(t_1) \subseteq \text{Set}(t_2)$. The advantages of semantic subtyping have been explored in [5, 7, 16], and subsequent work extended it from several perspectives: subtyping relation computation [4], decidability [18], and application to other programming paradigms [2, 29]. Singleton, union, and intersection types have received further attention through their set-based interpretation [11, 13]. MLstruct [28] uses a Boolean algebra of types to infer principal types with union and intersection for an ML-like language, handling subtyping that traditional Hindley-Milner inference avoids. This approach was later extended to dynamically-typed languages [8].

Hoare Logic for Type Safety. Hoare logic has also been used to ensure type safety in selected domains: typed assembly for operating systems [20, 36] and dynamically-typed programs [14, 15]. These works extract type information from the precondition and reduce type safety to verifying the Hoare triple $\{p\}e\{\text{true}\}$. Our work greatly expands on this approach, extending it with separation types, type predicates, case specifications, and a disciplined error hierarchy.

Type Systems for Memory and Resource Safety. Sub-structural type systems have been developed to reason about system resources such as memory, files, and locks [30]. The most prominent example is Rust, which uses ownership and borrowing to ensure memory safety. OCaml, a variant of OCaml, has similarly been extended with a linear type system to reason about affinity, uniqueness,

²Nevertheless, there is a recent retrofit of liquid types to Rust [24] which utilizes the ownership and borrowing mechanisms of Rust.

and locality of resources, supporting both memory safety and data race freedom [17, 25]. Our goal is similar, but our approach differs: rather than designing a new type discipline, we leverage the expressivity of Hoare logic to reason about memory and resource properties through separation types and flow-sensitivity.

From Uniqueness to Separation Type. Resource and ownership types for strong aliasing and flow-sensitivity have a rich prior art. A recent work [3] related frame rules in uniqueness type systems to separation logic via an FFI. Earlier, [31, 35] used alias types in typed assembly to describe heap shapes. Ownership types were subsequently used to support strong updates in refinement type systems [34]. Kloos et al. [23] combine liquid types with concurrent separation logic to track heap ownership across asynchronous tasks, supporting strong updates to heap location types. Similarly, Flux [24] lifts liquid-style refinements to Rust by exploiting Rust’s ownership mechanisms to enable strong updates. We introduce ownership in the sequential setting enabling flow-sensitive type mutation, and use first-class logical assertions together with type predicates rather than relying on a host language’s borrow checker, making our approach applicable to languages without built-in ownership and additionally supporting must-aliasing via singleton types. Our wider contribution is to unify these ideas within a single Hoare logic framework, increasing coverage of type-safety scenarios compared to any individual prior system.

Stability and Rely-Guarantee. The stability condition on pure type predicates (Definition 3.1) is the sequential counterpart of interference freedom in Owicki and Gries’s proof system for concurrent programs [27], and of stability under the rely relation in Jones’s rely-guarantee framework [21]. The closest type-system counterpart is the rely-guarantee reference system of [19], which enforces stability of refinement predicates over aliased mutable data via per-reference rely/guarantee annotations; our notion of stable pure type achieves the same guarantee in the sequential setting without such annotations.

9 CONCLUSION

This paper has presented a Hoare logic framework for type-safety verification that unifies separation types, case specifications, type predicates, and a disciplined error hierarchy. Building on semantic subtyping, we have extended the logic of types with: separation types that support flow-sensitive type mutation and must-aliasing; a comprehensive error hierarchy distinguishing Err (runtime errors), Exc (checked and unchecked exceptions), and Abrt (compile-time errors); path-sensitive and flow-sensitive type specifications via case specifications; and type predicates that subsume GADTs and liquid types. We have formalised the Hoare rules in Rocq and proved soundness, implemented a prototype verifier TypeHL in OCaml, and evaluated it on a benchmark suite demonstrating cases that standard type systems such as OCaml cannot handle.

10 DATA AVAILABILITY STATEMENT

Two artifacts accompany this paper. Firstly, a Rocq formalisation (described in Section 5.3) of our Type Safety via Hoare logic that was built on top of Separation Logic Foundations library[9]. This is mostly to prove soundness of our framework. Secondly, a lightweight implementation for TypeHL in OCaml to support automated checking of typed pre/post specifications (as described in Section 6.1). The Rocq formalisation and the implementation have been uploaded to <https://zenodo.org/records/19067445>. We intend to submit both for Artifact Evaluation.

REFERENCES

- [1] Rocq proof for soundness. [packaged-in-our-submitted-zip-file](#).

- [2] Davide Ancona and Andrea Corradi. Semantic subtyping for imperative object-oriented languages. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016*, pages 568–587. ACM, 2016.
- [3] Pilar Selene Linares Arévalo, , Arthur Azevedo de Amorim, Vincent Jackson, Liam O’Connor, Peter Schachte, and hristineRizkallah. Memory safety: Uniqueness as separation. In *Programming Languages and Systems - 23rd Asian Symposium, APLAS 2025*, volume 16201 of *Lecture Notes in Computer Science*, pages 3–21. Springer, 2025.
- [4] Gavin M. Bierman, Andrew D. Gordon, Catalin Hritcu, and David E. Langworthy. Semantic subtyping with an SMT solver. In *Proceeding of the 15th ACM SIGPLAN international conference on Functional programming, ICFP 2010*, pages 105–116. ACM, 2010.
- [5] Giuseppe Castagna. Semantic subtyping; Challenges, perspectives, and open problems. In *Theoretical Computer Science, 9th Italian Conference, ICTCS 2005*, volume 3701 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2005.
- [6] Giuseppe Castagna. Programming with union, intersection, and negation types. In Bertrand Meyer, editor, *The French School of Programming*, pages 309–378. Springer, 2024.
- [7] Giuseppe Castagna and Alain Frisch. A gentle introduction to semantic subtyping. In *Proceedings of the 7th International ACM SIGPLAN Conference on Principles and Practice of Declarative Programming*, pages 198–199. ACM, 2005.
- [8] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyen. Polymorphic type inference for dynamic languages. *Proc. ACM Program. Lang.*, 8(POPL):1179–1210, 2024.
- [9] Arthur Charguéraud. *Separation Logic Foundations*, volume 6 of *Software Foundations*. Electronic textbook, 2024. Version 2.2, <http://softwarefoundations.cis.upenn.edu>.
- [10] Wei-Ngan Chin, Cristina David, Huu Hai Nguyen, and Shengchao Qin. Automated verification of shape, size and bag properties via user-defined predicates in separation logic. *Science of Computer Programming*, 77:1006–1036, 2012.
- [11] Bruno C. d. S. Oliveira, Zhiyuan Shi, and João Alpuim. Disjoint intersection types. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016*, pages 364–377. ACM, 2016.
- [12] Stephen Dolan and Alan Mycroft. Polymorphism, subtyping, and type inference in MLsub. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017*, pages 60–72. ACM, 2017.
- [13] Jana Dunfield. Elaborating intersection and union types. In Peter Thiemann and Robby Bruce Findler, editors, *ACM SIGPLAN International Conference on Functional Programming, ICFP’12, Copenhagen, Denmark, September 9-15, 2012*, pages 17–28. ACM, 2012.
- [14] Björn Engelmann and Ernst-Rüdiger Olderog. A sound and complete Hoare logic for dynamically-typed, object-oriented programs. In *Theory and Practice of Formal Methods - Essays Dedicated to Frank de Boer on the Occasion of His 60th Birthday*, volume 9660 of *Lecture Notes in Computer Science*, pages 173–193. Springer, 2016.
- [15] Björn Engelmann, Ernst-Rüdiger Olderog, and Nils Erik Flick. Closing the gap - formally verifying dynamically typed programs like statically typed ones using Hoare logic - extended version. *CoRR*, abs/1501.02699, 2015.
- [16] Alain Frisch, Giuseppe Castagna, and Véronique Benzaken. Semantic subtyping. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*, pages 137–146. IEEE, 2002.
- [17] Aina Linn Georges, Benjamin Peters, Laila Elbeheiry, Leo White, Stephen Dolan, Richard A. Eisenberg, Chris Casinghino, François Pottier, and Derek Dreyer. Data race freedom à la mode. *Proc. ACM Program. Lang.*, 9(POPL):656–686, 2025.
- [18] Nils Gesbert, Pierre Genevès, and Nabil Layaïda. A logical approach to deciding semantic subtyping. *ACM Trans. Program. Lang. Syst.*, 38(1):3:1–3:31, 2015.
- [19] Colin S. Gordon, Michael D. Ernst, and Dan Grossman. Rely-guarantee references for refinement types over aliased mutable data. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 73–84, Seattle, WA, USA, June 2013. ACM.
- [20] Nadeem Abdul Hamid and Zhong Shao. Interfacing Hoare logic and type systems for foundational proof-carrying code. In *Theorem Proving in Higher Order Logics, 17th International Conference, TPHOLs 2004*, volume 3223 of *Lecture Notes in Computer Science*, pages 118–135. Springer, 2004.
- [21] Cliff B. Jones. Development methods for computer programs including a notion of interference. Technical Report PRG-25, Oxford University Computing Laboratory, 1981.
- [22] Ming Kawaguchi, Patrick Maxim Rondon, and Ranjit Jhala. Type-based data structure verification. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 304–315, Dublin, Ireland, June 2009. ACM.
- [23] Johannes Kloos, Rupak Majumdar, and Viktor Vafeiadis. Asynchronous liquid separation types. In *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, pages 396–420. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015.
- [24] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. Flux: Liquid types for Rust. *Proceedings of the ACM on Programming Languages*, 7(PLDI):169, June 2023.
- [25] Anton Lorenzen, Leo White, Stephen Dolan, Richard A. Eisenberg, and Sam Lindley. Oxidizing OCaml with modal memory management. *Proc. ACM Program. Lang.*, 8(ICFP):485–514, 2024.

- [26] Meta. The Hack programming language. <https://hacklang.org>, 2014.
- [27] Susan S. Owicki and David Gries. An axiomatic proof technique for parallel programs I. *Acta Informatica*, 6(4):319–340, 1976.
- [28] Lionel Parreaux and Chun Yin Chau. MLstruct: principal type inference in a boolean algebra of structural types. *Proc. ACM Program. Lang.*, 6(OOPSLA2):449–478, 2022.
- [29] Tommaso Petrucciani, Giuseppe Castagna, Davide Ancona, and Elena Zucca. Semantic subtyping for non-strict languages. In *24th International Conference on Types for Proofs and Programs, TYPES 2018*, volume 130 of *LIPICs*, pages 4:1–4:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- [30] Benjamin C. Pierce. *Advanced Topics in Types and Programming Languages*. MIT Press, 2004.
- [31] Frederick Smith, David Walker, and J. Gregory Morrisett. Alias types. In Gert Smolka, editor, *Programming Languages and Systems, 9th European Symposium on Programming, ESOP 2000, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS 2000, Berlin, Germany, March 25 - April 2, 2000, Proceedings*, volume 1782 of *Lecture Notes in Computer Science*, pages 366–381. Springer, 2000.
- [32] Jane Street. <https://oxcaml.org/>, 2025.
- [33] Sam Tobin-Hochstadt and Matthias Felleisen. The design and implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 395–406. ACM, 2008.
- [34] John Toman, Ren Siqui, Kohei Suenaga, Atsushi Igarashi, and Naoki Kobayashi. Consort: Context- and flow-sensitive ownership refinement types for imperative programs. In *ESOP*, volume 20, pages 684–714, 2020.
- [35] David Walker and J. Gregory Morrisett. Alias types for recursive data structures. In Robert Harper, editor, *Types in Compilation, Third International Workshop, TIC 2000, Montreal, Canada, September 21, 2000, Revised Selected Papers*, volume 2071 of *Lecture Notes in Computer Science*, pages 177–206. Springer, 2000.
- [36] Jean Yang and Chris Hawblitzel. Safe to the last instruction: automated verification of a type-safe operating system. In *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010*, pages 99–110. ACM, 2010.

A BASIC FEATURES OF TYPES

A.1 Wider Usage of Singleton Types

Recently, singleton types were used in [8, 28] for constant literals of the form c , such as 3 or "hi". Whenever a variable v is bound to a constant c , we use the type notation $v : c$ to state that v has singleton type c .

In Scala, a singleton object class is essentially a class with a single instance. In this paper, we define a singleton type as a type containing only a single value. We extend singleton types to include parameter (input) variables such as x , using the annotation $r : x$ to denote that result r has singleton type x . In Hoare logic terms, $r : x$ is equivalent to $r = x$, since any output r with singleton type x has the same value as x .

An example is the identity function:

```
id x = x
```

Using the singleton type, we write $id : (x : Any \rightarrow r : x)$, where the input has type Any and the output has singleton type x . This requires fewer universally quantified type variables than the principal type $id : (\forall a. a \rightarrow a)$. One can verify that $id : (x : Any \rightarrow r : x)$ is equivalent to $id : (\forall a. a \rightarrow a)$; the former has fewer quantified variables and is therefore easier to apply during type-checking (requiring fewer existential instantiations). In Hoare logic notation, this corresponds to $id :: (\lambda x r. \text{req } true \text{ ens}[r] r : x)$.

As a second example, consider $\lambda x. x x$, where the parameter x is applied to itself. Using singleton types, its function type is $\forall b. (x : x \rightarrow b) \rightarrow b$, which uses one fewer quantified variable than the $\forall a, b. (a \wedge a \rightarrow b) \rightarrow b$ formulation of [8]. Fewer quantified variables simplify existential instantiation for polymorphic method calls. Using our formulation, one can prove $(\lambda x. x x)(\lambda x. x x) : \perp$ and $(\lambda x. x x)(\lambda x. x) : (x : Any \rightarrow r : x)$, both of which are beyond the reach of [8]. The application $(\lambda x. x x)(3)$ is correctly rejected as untypeable by both [8] and our framework. We currently restrict singleton types to variables and function-call arguments; field positions use type variables instead.

A.2 Constructors and Type Predicates

In statically-typed languages such as OCaml, algebraic data types are standard. Using the polymorphic list type from Sec. 1, the type annotation for `length` is:

```
(* Type signature and implementation of length *)
length : ∀T. List(T) → Int
length(xs) = match xs of
  {Nil -> 0;
   Cons(y,ys) -> 1 + length(ys)}
```

Our framework supports types that are more fine-grained than declared algebraic data types. The type $List(T)$ is semantically equivalent to the recursive predicate:

```
r : List(T) ≡ r:Nil ∨ r:Cons(T, List(T))
```

Under semantic subtyping, Nil and $Cons$ are subtypes of $List$ and can appear directly in specifications. A more precise specification for `length` is:

```
(* Type specification of length *)
length : ∀T. case[xs]
  {Nil ens[r] r:0;
   Cons(T, List(T)) ens[r] r:Int}
```


Here $r : 0$ is a singleton type, since $Set(\{0\}) \subset Set(Int)$. Singleton return types enable more informative specifications for path-sensitive code, particularly when values with distinct constructors produce different outcomes.

Recursive type predicates support algebraic data types in static typing scenarios and can be extended to GADTs (Sec. 2.4) and type classes (Appendix B.1).

Although recursive predicates define stronger types, they may appear incompatible with dynamically-typed programs that rely on runtime type testing. For complex types such as $List(List(T))$, we propose using the unnested form $List(-)$ for runtime testing. Concretely, $List(-)$ abbreviates $Nil \vee Cons(-, -)$, as described next.

A.3 Combining Type Testing with Pattern-Matching

Statically-typed languages use pattern-matching; dynamically-typed languages rely on type-testing. Recent proposals, including C#, attempt to unify both. We achieve this integration via an enhanced match construct that supports both pattern-matching and structural type-testing.

The function `length_generic` dispatches the length computation based on the runtime type of its argument. Its case specification is:

```
(* Type spec and code of length *)
length_generic : VT. case[xs]
  {xs:Int   ens[r] r:1;
   xs:Str   ens[r] r:Int;
   xs>List(T) ens[r] r:Int}
length_generic xs = match xs of
  {Int -> 1;
   Str -> length_of_str(xs);
   List(_) -> length(xs)}
```

Recall that $List(-) \equiv Nil \vee Cons(-, -)$. The case specification with singleton type 1 for the *Int* branch is strictly more precise than the intersection type:

```
length_generic : (VT. List(T) → Int) ∧
  (Int → Int) ∧ (Str → Int)
```

The method `length` was already defined in Sec. A.2.

B ADVANCED TYPES

B.1 Support for Type Classes

We describe how our framework supports type classes, as adopted by Haskell and Scala for ad hoc polymorphism. The `Num` type class supports generic numeric operations:

```
+ : VT · Num(T) ⇒ T → T → T
```

Using pre/post type specifications, we encode this with a type-class predicate $Num(T)$ built from the instance declarations of `Num`:

```
Num(T:*) ≡ (T=Int) ∨ (T=Float) ∨ ...
```

This predicate appears as a precondition constraint on overloaded operations:

```
+ :: λ x y r · ∀ T · req[x,y] x:T ∧ y:T
  ∧ Num(T) ens[r] r:T
```

Higher-order type classes are supported similarly. For the `Functor` class:

```

class Functor(F:*→*) {
  map : ∀A,B · (A→B) → F(A) → F(B)

```

the specification for map is:

```

map :: λ f xs r · ∀ F:*→*,A,B ·
  req[f,xs] f:(A→B) ∧ xs:F(A) ∧ Functor(F)
  ens[r] r:F(B) Functor(F:*→*) ≡ (F=List) ∨ (F=Option) ∨ ...

```

Type classes may be organised in an inheritance hierarchy. For example, since *Monad* extends *Functor*, we have the lemma $\forall F : (* \rightarrow *) . \text{Monad}(F) \Rightarrow \text{Functor}(F)$. Note that type-class predicates such as *Num*($_$) and *Functor*($_$) may not be used for runtime type-testing, since they range over types rather than values.

B.2 Gradual Typing for Dynamic Languages

Although our expressive specifications capture many dynamically-typed programs as statically safe, some examples remain outside the current framework. Consider:

```

f(x, y) = if x > 5 then !y else y + 1

```

For this to be statically type-safe, the precondition would need $y : \text{Ref}(T) \wedge \text{Int}$, which reduces to $y : \perp$, making the precondition unsatisfiable. Gradual typing addresses this via a special family of pure types $?t$, allowing runtime casts. We permit $y : ?(\text{Ref}(T) \vee \text{Int})$ in specifications:

```

req[x, y] x : Int ∧ y : ?(Ref(T) ∨ Int)
ens[r] (y : Ref(T) ∧ r : T) ∨ (y : Int ∧ r : Int)

```

Runtime casts are inserted into the program body to enforce the chosen branch:

```

f(x, y) = if x > 5
  then let z = (?Ref(_)) y in !z
  else let z = (?Int) y in z + 1

```

Such programs are gradually typed: some execution paths may produce a runtime abort. The runtime cast $(?p) v$ is defined as:

```

(?p) v = match v of { p -> v; _ -> abort }

```

The Hoare rule for runtime cast ignores abort, treating it as a runtime error rather than a compile-time error:

$$\frac{\text{fresh } r \quad \Delta \vdash x : ?(_) \quad \Delta_1 \equiv \Delta \wedge (x : p) \wedge r : x}{\{ \Delta \} (?p) x \{ \text{ens}[r] \Delta_1 \}} \quad \text{R-CAST}$$

B.3 Restrictions to Support Decidability

In their full generality, type predicates are not decidable — arbitrary recursive predicates with unrestricted quantification and negation quickly exceed the reach of automated verification. In practice, decidability can be recovered by imposing the following five restrictions, which together cover the vast majority of data structure predicates that arise in verification.

- Structural recursion only. Predicates must unfold only on the direct sub-components of a constructor — no mutual recursion through non-structural positions. This makes well-foundedness syntactically checkable and ensures unfolding always terminates.
- Linear arithmetic for index constraints. Restricting index constraints to linear arithmetic over integers (e.g. $n = m + 1, \text{lo} \leq v$) reduces subtyping obligations to linear arithmetic validity, decidable by standard SMT solvers via the theory of linear integer arithmetic (LLA). Non-linear arithmetic or quantifier alternation would push into undecidability.

- Restricted existential quantification. Existentials in predicate bodies must be eliminable — either by unification during unfolding, or by bounding them over finite domains. Unrestricted nested existentials can render subtyping undecidable.
- Negation-free or negation at base types only. Boolean combinations $\vee$ and $\wedge$ are decidable provided sub-predicates are decidable. However, negation ($\neg$) of recursive predicates introduces co-inductive reasoning and is generally undecidable. Restricting negation to base types only preserves decidability.
- Symbolic heap fragment for separation predicates. Full separation logic with arbitrary recursive heap predicates is undecidable. The symbolic heap fragment recovers decidability by restricting the form of inductive predicate definitions rather than banning recursion itself: each predicate case must be expressed using only points-to assertions, separating conjunctions, inductive predicate calls, and pure linear arithmetic side conditions — with no negation and no non-separating disjunction at the heap level. Recursive inductive predicates such as $x \mapsto \text{List}(T)$, $x \mapsto \text{Tree}(T)$, and list segment predicates are fully expressible within this fragment, and bi-abduction over such predicates is decidable — as exploited by tools such as Smallfoot.

These five restrictions together form the practical sweet spot adopted by most separation logic verifiers: structural recursion, linear arithmetic indices, eliminable existentials, negation-free recursive predicates, and symbolic heap separation. This covers $\text{List}(T, n)$, $\text{RBTREE}(T, c, h)$, $\text{SortedList}(T, lo, hi)$ and the data structure predicates that arise throughout this paper, while keeping verification tractable via SMT solving.

C EXCEPTION HANDLING

The Abt or $\top$ types are nearly always avoided in the result type of our functions, as any encounter with abort will cause our program to fail immediately. Case specification is the only type specification which allows Abt or $\top$ to appear in the *ens* clauses of the otherwise (i.e. $_$) clause, due to the need for completeness in case specifications.

Runtime errors are often modelled as exceptions that could be handled by suitable error recovery routines. In some languages, such as Java, they are further classified as *checked exceptions* which must be flagged by the type system or *unchecked exceptions* whose occurrences can be ignored by the type system.

In our solution, we would classify checked exceptions (denoted by ChExc) and unchecked exceptions (denoted by UnExc) as subtypes of Exc :

$$\perp <: \{\text{ChExc}, \text{UnExc}\} <: \text{Exc} <: \top$$

Additionally, UnExc is a subtype of valid types and ChExc is disjoint from both Any and Abt :

$$\begin{aligned} \text{UnExc} &<: \{..valid types..\} \wedge \\ ((\text{ChExc} \wedge \text{Any}) &= (\text{ChExc} \wedge \text{Abt}) = \perp) \end{aligned}$$

In this way, checked exceptions will always be carefully tracked by our type system, while unchecked exceptions are considered as a subtype of various valid types, and can be omitted in our type specifications. We recall this subtype relation in our lattice of types in Fig 1, where $..valid types..$ include all standard types, e.g. Int , Str , $\text{List}(\text{Int})$, and their separation counterparts.

Try-catch handling can now be implemented via a let-binding, followed by type testing. Raised exceptions that pass type-casting are propagated outwards via *match* construct, as illustrated by the second translation rule below.

```

1422      try e1 catch Exc -> e2
1423      ⇔ let x = e1 in match x {Exc -> e2 | _ -> x}
1424
1425      let x:t = e1 in e2
1426      ⇔ let x = (t ∨ Exc) e1 in
1427      match x {Abt ∨ Exc -> x | _ -> e2}
1428
1429

```

1430 D MORE ON SEPARATION TYPES

1431 D.1 Uniqueness vs Separation Type

1432 Each uniqueness type $x \mapsto T@U$ is mutually disjoint from both the separation type $x \mapsto T$ and the
 1433 pure type $x : T$, as the following contradictions show ($\Leftrightarrow$ denotes logical equivalence):
 1434

$$1435 \quad x \mapsto T@U \wedge x : T \Leftrightarrow x \mapsto T \wedge x : T \Leftrightarrow x \mapsto T * x \mapsto T@U \Leftrightarrow \text{false}$$

1437 This design choice ensures each type form has distinct capabilities. Uniqueness and separation
 1438 types are interconvertible via the following equivalences:
 1439

$$1440 \quad x \mapsto T@U \Leftrightarrow x \mapsto T \wedge \text{NoAlias}(x)$$

$$1441 \quad \text{NoAlias}(x) \wedge x = y \Leftrightarrow \text{false}$$

1443 Every uniqueness type converts to a separation type; each separation type converts to a uniqueness
 1444 type by dropping all aliases of the form $x = y$.

1445 As type-safety is based on over-approximation, the following weakening lemmas apply:

$$1446 \quad x \mapsto T@U \wedge T <: A \Rightarrow_{\text{weaken}} x \mapsto A@U$$

$$1447 \quad x \mapsto T \wedge T <: A \Rightarrow_{\text{weaken}} x \mapsto A$$

$$1448 \quad x \mapsto T \Rightarrow_{\text{weaken}} x : T$$

1451 Types are always initialised as strong as possible and weakened only when necessary. Separation and
 1452 uniqueness types are assigned at construction and may be irreversibly weakened to flow-insensitive
 1453 pure types (last lemma above).
 1454

1455 Both allow type mutation for flow-sensitivity; however, separation types additionally permit
 1456 must-aliases, whereas uniqueness types allow only a single reference – making separation types
 1457 strictly more expressive. For example, uniqueness types cannot handle $\text{swap}(m, m)$ since the two
 1458 arguments must be unaliased.

1459 D.2 Separation Type for Mutable Data Structure

1460 For pure types, inductive predicates capture the weakest property guaranteed for a given algebraic
 1461 data type. For $\text{List}(T)$:
 1462

$$1463 \quad r : \text{List}(T) \equiv r : \text{Nil} \vee (\exists q. r : \text{Cons}(T, q) \wedge q : \text{List}(T))$$

1465 This predicate is stable for every pure $\text{List}(T)$ value. Both acyclic and cyclic lists are instances:

$$1466 \quad x : \text{Cons}(T, y) \wedge y : \text{Cons}(T, z) \wedge z : \text{Nil} \vdash x : \text{List}(T)$$

$$1467 \quad x : \text{Cons}(T, y) \wedge y : \text{Cons}(T, z) \wedge z : x \vdash x : \text{List}(T)$$

Using separation types, acyclic and cyclic lists can be distinguished by the following separation predicates:

$$x \mapsto \text{AList}(T) \Leftrightarrow x : \text{Nil} \vee \exists q. x \mapsto \text{Cons}(T, q) * q \mapsto \text{AList}(T)$$

$$x \mapsto \text{CList}(T) \Leftrightarrow x \mapsto \text{Cons}(T, q) * q \mapsto \text{LSeg}(T, x)$$

$$x \mapsto \text{LSeg}(T, p) \Leftrightarrow x : p \vee \exists q. x \mapsto \text{Cons}(T, q) * q \mapsto \text{LSeg}(T, p) \wedge x : \neg p$$

The list-segment predicate $x \mapsto \text{LSeg}(T, p)$ captures an incoming pointer x and an outgoing pointer p , enabling circular list support. These predicates yield the following provable and disprovable assertions:

$$x \mapsto \text{Cons}(T, y) * y \mapsto \text{Cons}(T, z) \wedge z : \text{Nil} \vdash x \mapsto \text{AList}(T)$$

$$x \mapsto \text{Cons}(T, y) * y \mapsto \text{Cons}(T, z) \wedge z : x \vdash x \mapsto \text{CList}(T)$$

$$x \mapsto \text{Cons}(T, y) * y \mapsto \text{Cons}(T, z) \wedge z : \text{Nil} \not\vdash x \mapsto \text{CList}(T)$$

$$x \mapsto \text{Cons}(T, y) * y \mapsto \text{Cons}(T, z) \wedge z : x \not\vdash x \mapsto \text{AList}(T)$$

Separation predicates are flow-sensitive and express data structure invariants that type-safe verification must maintain. We propose guaranteeing these invariants via pre/post conditions expressed with appropriate separation predicates.

D.3 Separation Type for Data Race Freedom

A concurrent program has a data race if two threads access the same shared data, at least one access is a write, and there is no synchronisation ordering between them. One way to ensure race freedom is to guarantee disjoint memory access, expressed via separation types at field-level granularity.

Consider a mutable $\text{Pair}(X, Y)$ with fields of types X and Y and the following two concurrent threads:

$$p.\text{Pair}.1 := \text{"hi"} \parallel p.\text{Pair}.2 := p.\text{Pair}.2 + 1$$

Although both threads write to p , they access disjoint fields. The following field-splitting lemma supports this:

$$p \mapsto \text{Pair}(X, Y) \Leftrightarrow p.\text{Pair}.1 \mapsto X * p.\text{Pair}.2 \mapsto Y$$

A pair $p \mapsto \text{Pair}(X, Y)$ can be split into disjoint field ownership $p.\text{Pair}.1 \mapsto X * p.\text{Pair}.2 \mapsto Y$. Type-safe verification then proceeds as follows:

$$\{ p \mapsto \text{Pair}(\text{Any}, \text{Int}) \}$$

$$\{ p.\text{Pair}.1 \mapsto \text{Any} * p.\text{Pair}.2 \mapsto \text{Int} \}$$

$$p.\text{Pair}.1 := \text{"hi"} \parallel p.\text{Pair}.2 := p.\text{Pair}.2 + 1$$

$$\{ p.\text{Pair}.1 \mapsto \text{Str} * p.\text{Pair}.2 \mapsto \text{Int} \}$$

$$\{ p \mapsto \text{Pair}(\text{Str}, \text{Int}) \}$$

To ensure data race freedom, only separation types (not pure types) may be used in concurrent threads unless the data structure is immutable: pure types permit arbitrary aliases that other threads could mutate independently. When concurrent threads require write access to shared heap memory, the framework can be extended with a locking mechanism over separation-typed data structures.

E OPERATIONAL SEMANTICS

To facilitate the following soundness proofs, we define a big-step reduction relation with judgments $e, h, S \rightsquigarrow R_e, h_1, S_1$. Program states consist of a heap h and store S , like in Sec. 5. Outcomes are $R_e ::= \text{Norm}(v) \mid \text{Abt}$ where $\text{Norm}(v)$ captures the valid values and Err values which could occur during the run-time but do not terminate the programs. The Abt represents a failure which will

immediately kill the process. In the *Match* case, we assume there is an implicit case that handles a match failure. If the process cannot find a case to match, it will terminate the program with *Abtrt*.

$$\begin{array}{c}
\frac{}{v, h, S \rightsquigarrow v, h, S} \text{Nil, Const, Lambda} \\
\frac{}{x_1 :: x_2, h, S \rightsquigarrow S(x_1) :: S(x_2), h, S} \text{Cons} \\
\frac{}{x, h, S \rightsquigarrow S(x), h, S} \text{Var} \\
\frac{\text{type}(S(x)) <: t}{(t(x)), h, S \rightsquigarrow x, h, S} \text{Cast} \\
\frac{\text{type}(S(x)) \not<: t}{(t(x)), h, S \rightsquigarrow \text{Abtrt}, h, S} \text{Cast} \\
\frac{e_1, h, S \rightsquigarrow v, h_1, S_1 \quad e_2, h_1, S_1[x := v] \rightsquigarrow v_1, h_2, S_2}{(\text{let } x = e_1 \text{ in } e_2), h, S \rightsquigarrow v_1, h_2, S_2 \setminus \{x\}} \text{Let} \\
\frac{S(f) = \lambda x^*. \Phi e \quad S(x^*) = v^* \quad [x^* := v^*]e, h, S \rightsquigarrow v_2, h_1, S_1}{f(x^*), h, S \rightsquigarrow v_2, h_1, S_1} \text{App} \\
\frac{x_1 \in \text{dom}(h)}{(x_1 := x_2), h, S \rightsquigarrow (), h_1[S(x) := S(v)], S} \text{Assign} \\
\frac{}{!x, h, S \rightsquigarrow h(S(x)), h, S} \text{Deref} \\
\frac{l \notin \text{dom}(h)}{\text{ref } x, h, S \rightsquigarrow \ell, h[\ell := S(x)], S} \text{Ref} \\
\frac{S(x) \subseteq p_i \quad e_i, h, s \rightsquigarrow v, h_i, S_i}{(\text{match } x \{p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n\}), h, S \rightsquigarrow v, h_i, S_i} \text{Match}
\end{array}$$